PGP - AIML

FINAL REPORT

NLP CHATBOT

(Team: Capstone Group 1 - NLP 1)

TEAM MEMBERS

Aparana Sehgal

Onkar Lavhekar

Pramod Giri

Saravana Perumal

Siddharth Kulkarni

Problem Statement, Data and Findings **4**
Data Description 4
Project Objective 5
Data Analysis and Findings 5
Overview of the final process **6**
Pre-processing and EDA **7**
Pre-processing 7
EDA 8
Accident volumes, their industry and country-wide occurrences 8
Genders involved in the incidents 10
Incident Category 11
Incidents Occurrence 12
Vulnerable Worker Type to the Accidents 14
Accidents per Country 15
Accident Types for various Industries 16
Accidents in Countries based on Industry 17
Model Building and Evaluation **18**
Model Design 18
Models 18
Benchmarking - Comparison of Experiments **20**
Milestone 1 20
Model Part 1 - Traditional ML Algorithms 20
Model Part 1.1 Tokenizer + GloVe Without SMOTE 20
Model Part 1.2 Tokenizer + GloVe WITH SMOTE 29
Model Part 1.3 Transformers 37
Models Summary 46
Milestone 2 47
Model Part 1.4 Hypertune ML Models 47
Model Part 2 - Advanced ML Models 60
Model Part 2.1 Neural Network Models 60
Model Part 2.2 BiLSTM (Glove + Tokenizer) 63
Model part 2.3 BiLSTM (Sentence Transformer) 66
Models Summary 69
Cross Model Performance 70
Milestone 3 71
Chatbot UI 71
Summary 74

Implication - Business Values Derived **75**
Limitations **76**
Recommendations **77**
Closing Reflections **78**
Technical References: 78

Problem Statement, Data and Findings

The database comes from one of the biggest industries in Brazil and in the world. It is an urgent need for industries/companies around the globe to understand why employees still suffer some injuries/accidents in plants. Sometimes they also die in such an environment.

Data Description

Data: timestamp or time/date information

Countries: which country the accident occurred (anonymous)

Local: the city where the manufacturing plant is located (anonymous)

Industry sector: which sector the plant belongs to

Accident level: from I to VI, it registers how severe was the accident (I means not severe but VI means very severe)

Potential Accident Level: Depending on the Accident Level, the database also registers how severe the accident could have been (due to other factors involved in the accident)

Gender: if the person is male of female

Employee or Third Party: if the injured person is an employee or a third party

Critical Risk: some description of the risk involved in the accident

Description: Detailed description of how the accident happened.

Project Objective

Design a ML/DL/Transformer based chatbot utility which can help the professionals to highlight the safety risk as per the incident description.

Data Analysis and Findings

Data contains the HSE incidents which have happened in the 12 different plants and 3 different countries.

The below screenshot shows the glimpse of raw data.

	Unnamed: 0	Data	Countries	Local	Industry Sector	Accident Level	Potential Accident Level	Genre	Employee or Third Party	Critical Risk	Description
0	0	2016-01-01	Country_01	Local_01	Mining	I	IV	Male	Third Party	Pressed	While removing the drill rod of the Jumbo 08 f...
1	1	2016-01-02	Country_02	Local_02	Mining	I	IV	Male	Employee	Pressurized Systems	During the activation of a sodium sulphide pum...
2	2	2016-01-06	Country_01	Local_03	Mining	I	III	Male	Third Party (Remote)	Manual Tools	In the sub-station MILPO located at level +170...
3	3	2016-01-08	Country_01	Local_04	Mining	I	I	Male	Third Party	Others	Being 9:45 am. approximately in the Nv. 1880 C...
4	4	2016-01-10	Country_01	Local_04	Mining	IV	IV	Male	Third Party	Others	Approximately at 11:45 a.m. in circumstances t...

Insights/Findings:

1. Data set consists of 425 rows and 11 columns
2. There are no null values in the data set
3. Column "Unnamed: 0" seems to be the primary key in the data
4. Column "Data" has the dates and has datatype "datetime64[ns]"
5. Column "Local" has the cities data
6. Column "Genre" has the gender data
7. All columns except "Unnamed" and "Data" have object datatype

Overview of the final process

The brief approach for the solution explained below

1. Solution requires the model building to classify different possible HSE incident accident severity levels.
2. Data cleansing and Pre-Processing are important to have a good cleaned input dataset for the model to predict the expected output. Hence the data cleansing and pre-processing steps are performed in a detailed manner.
3. Plot visualizations to understand the data relationships and patterns. Also, helps to understand the patterns in the data.
4. Team will try different approaches for the model building and compare the models.
    a. Glove + Dummies and run the different classifiers
    b. Apply SMOTE on the data from Glove and Dummies and run the different classifiers.
    c. Create Embedding using the Sentence Transformer, and run using different classifiers.
    d. Tune the parameters on the classifiers used on the SMOTE and run the classifiers again with hyperparameters.
    e. Train the Neural Network with the Sentence Transformer output.
    f. Train the BiLSTM model using the Glove + Tokenizer
    g. Train the BiLSTM model using the Sentence Transformer.
5. The benchmarking of outcome has been captured. The performance of the model is tuned based on the different iterations with different parameters.
6. The business derived value based on the outcome of the model is analyzed.
7. Limitations of the model and scope of improvements has to be identified and documented.
8. Documenting the lessons learned on each of the steps of the project is duly noted and summary is created as learning and future reference.

Pre-processing and EDA

Pre-processing

The data needs to be cleaned up before it can be analyzed and used for the modeling.

Preprocessing Steps:

1. Null Check : Check if no missing data in the dataset
2. Drop Columns : Drop unnecessary columns which does not help in model building
3. Rename Columns : For better visualization and understanding, rename existing columns. This step is completely optional.
4. Unique Check : Check the data uniqueness and replace some irrelevant data with appropriate values.
5. Duplicate Check : Remove the duplicate records for better model training and to save computational time.
6. Feature Creation : This is purely domain specific work. Feature creation involves creating new feature from existing features available in dataset
7. Save Checkpoint : Save Preprocessed data to resume the work from the same checkpoint. This is an optional task but an important one.

Actions :

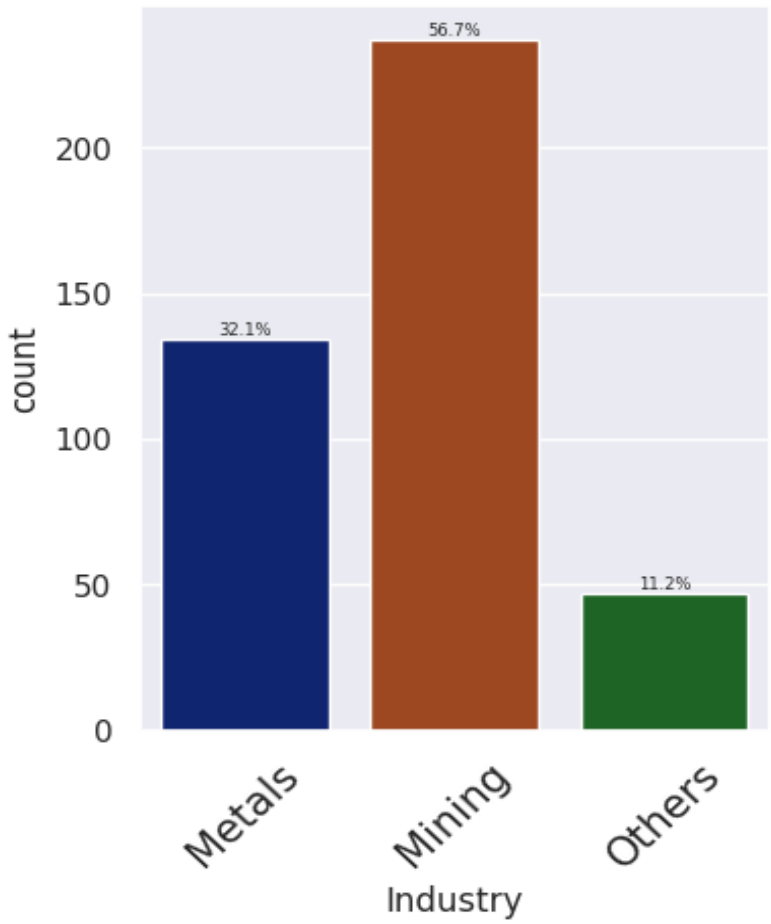
1. There are no null values in the data set
2. Column "Unnamed:0" has been dropped as it has no impact on model building
3. The data contains 7 duplicates and which brings the row count of dataset to 418 after removing the duplicates
4. The Category Risk column contains the value ‘\nNot applicable’ which needs to be replaced with category ‘Others. This brings the count of ’Others’ category to 230.

EDA

There can be a number of plots being analyzed to identify trends and other valuable insights.

Accident volumes, their industry and country-wide occurrences

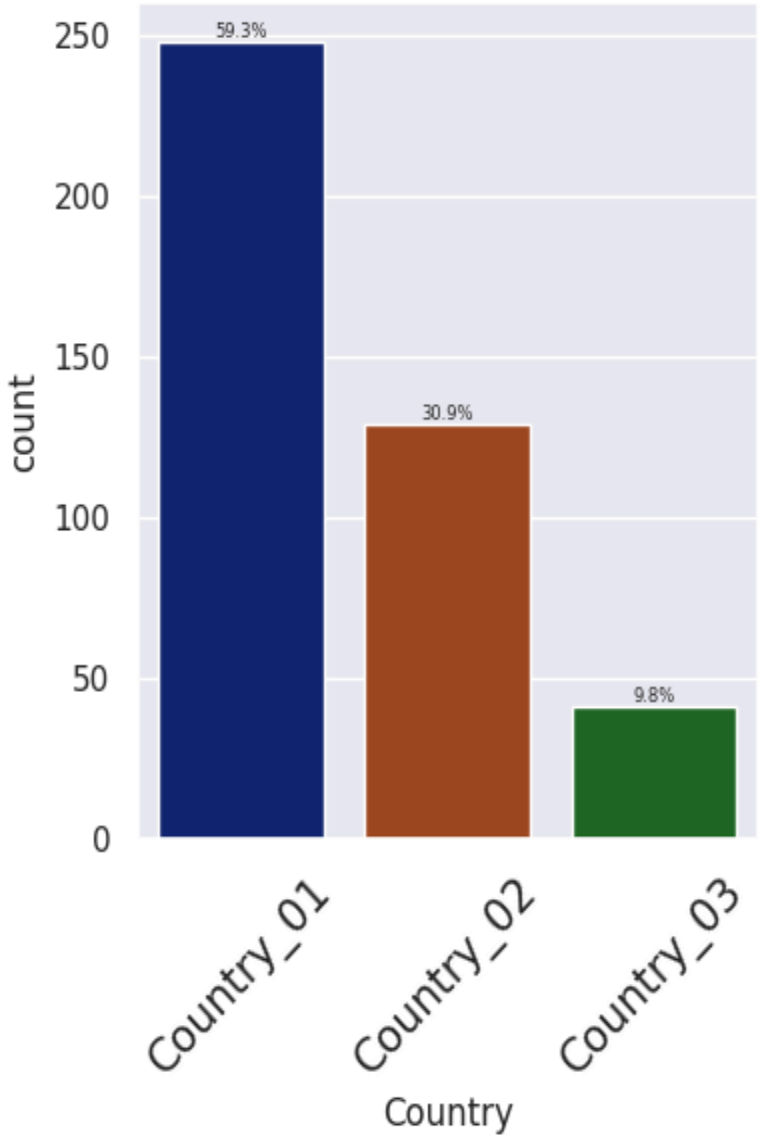
Industry break-up:



Observations:

- Almost 57% or 237 records of incidents are in the mining industry.
- This is followed by approx. 32% or 134 records of incidents in the metals industry.
- Other industries constitute roughly 11% or 47 records of incidents.

Country break-up:



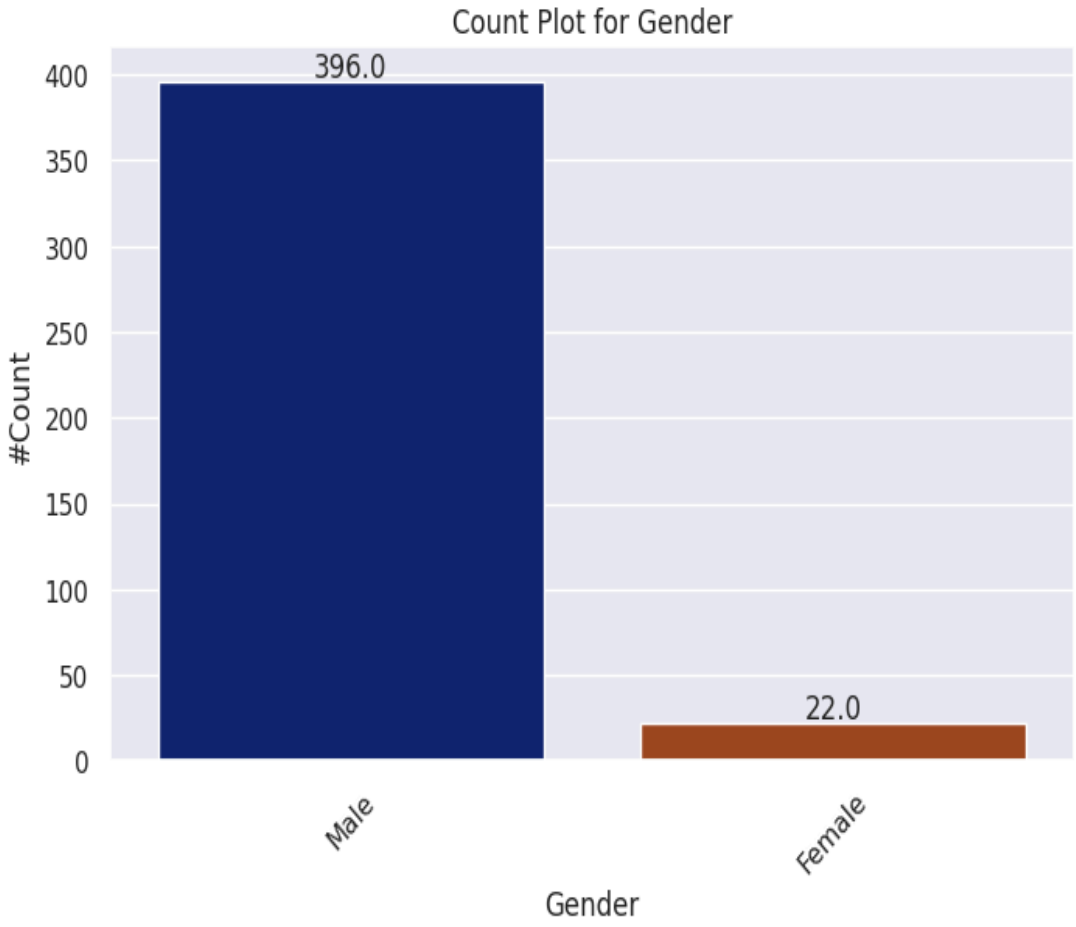
Observations:

- Country 1 has more than 59% or 248 records of accidents.
- This is followed by approx. 31% or 129 records for Country 2.
- Country 3 has about 10% or 41 records.

Recommendation: An appropriate training program concentrating the country, industry and the accident level should help reduce the accidents.

Genders involved in the incidents

Gender-wise break-up:

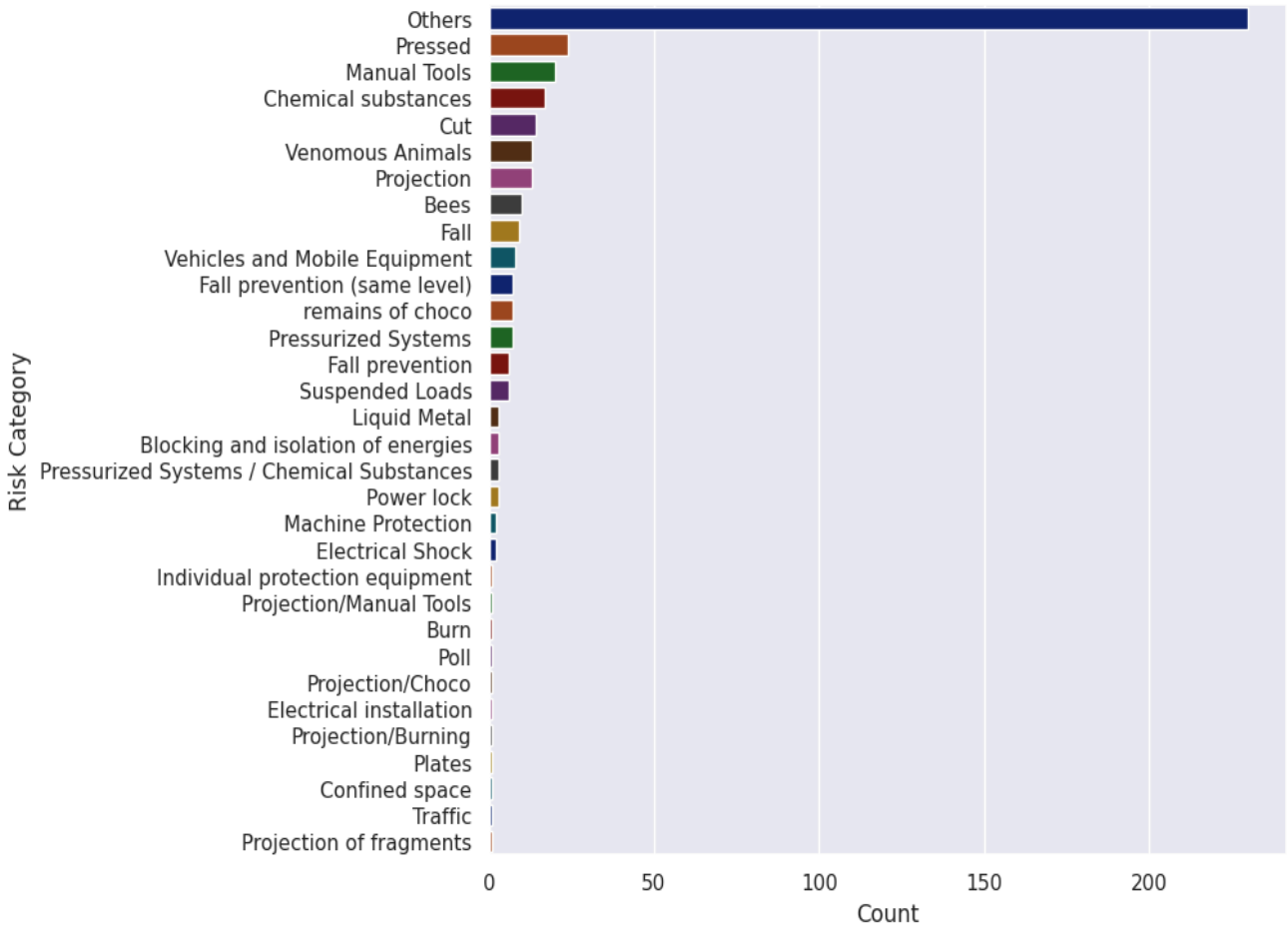


Observations:

- The data shows that there are 396 Males involved in the incidents
- Whereas the count of incidents recorded for females is only 22
- We can infer that accidents were more likely for one gender over the other

Incident Category

Incident Category-wise break-up:

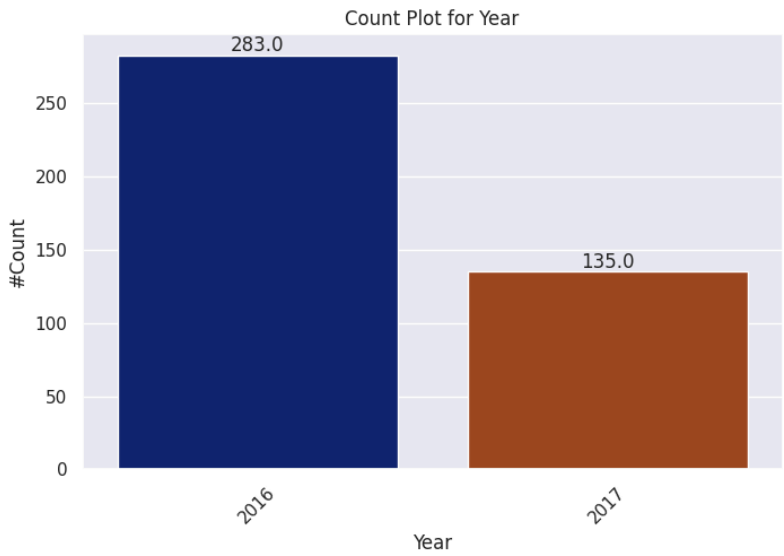


Observations:

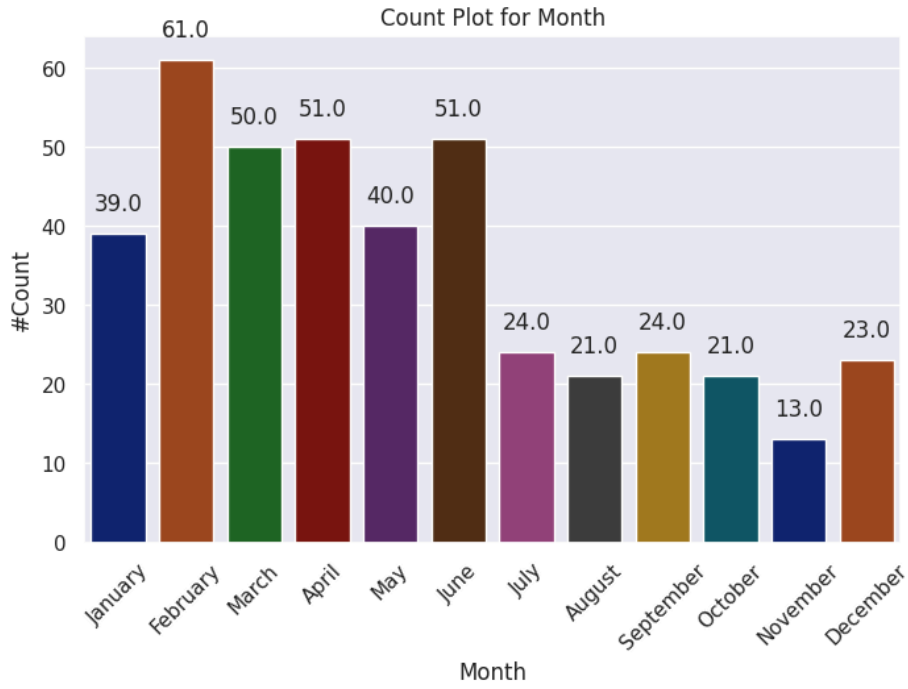
- As per the data maximum incidents falls under category 'Other'. The Count of Risks for the Others category is 230.
- Pressed, Manual Tools, Chemical substances are the top most issues in specified risk categories.

Incidents Occurrence

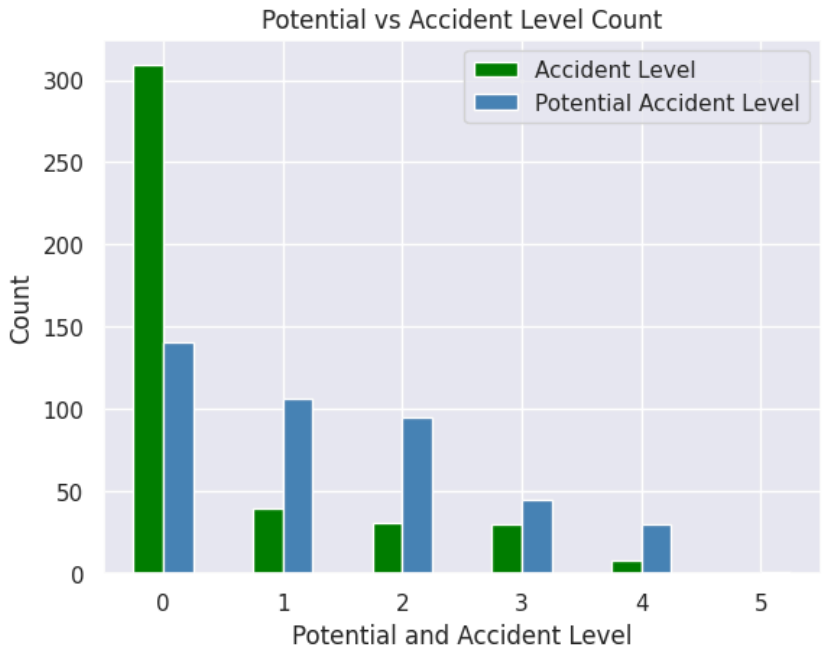
Yearly break-up:



It seems there have been more incidents in 2016 than 2017. Looking at the data maximum incidents seems to be in February (61) followed by April and June (51 each).



The data suggests that the Potential Accident level is higher than the Accident levels reported due to other factors in “Level 0”. Also, the graph indicates the drop in Potential Accident Level of 0 Severity and increase in others i.e. many of Severity 0 incidents got updated to Potential Severity 1.

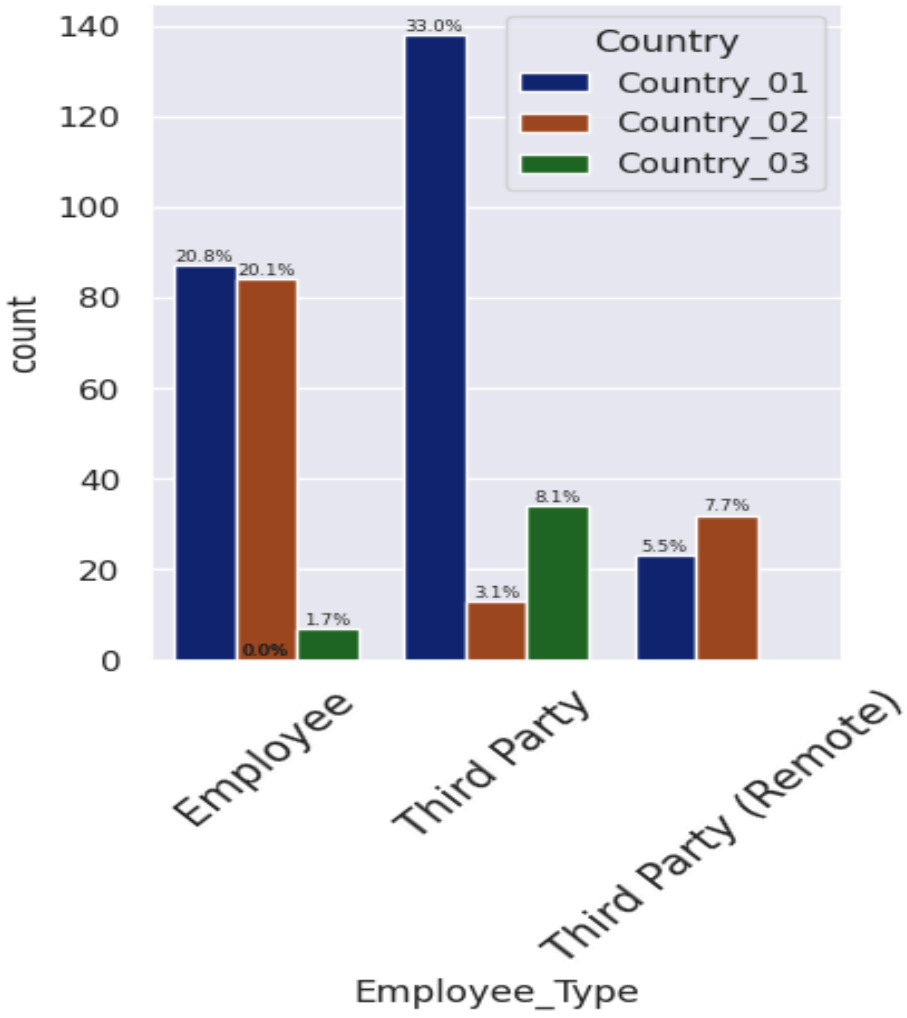


Observations:

- It may seem that accidents reduced in 2017 when compared with 2016 as the count for 2017 is 135 while for 2016 it is 283.
- However, the data is only available7 months in 2017.
- Same conclusion can be derived from the monthly graphs.
- Accidents from Jan to July have almost double the values from Aug to Dec as the latter half of the year only has data for 2016, while the 7 months have data for 2 years.
- The potential accident level for accident level 0 is projected less than half of its actual value.
- While for all other categories the accident level is less than half of the potential accident level projected.

Vulnerable Worker Type to the Accidents

Country break down for each employee type:

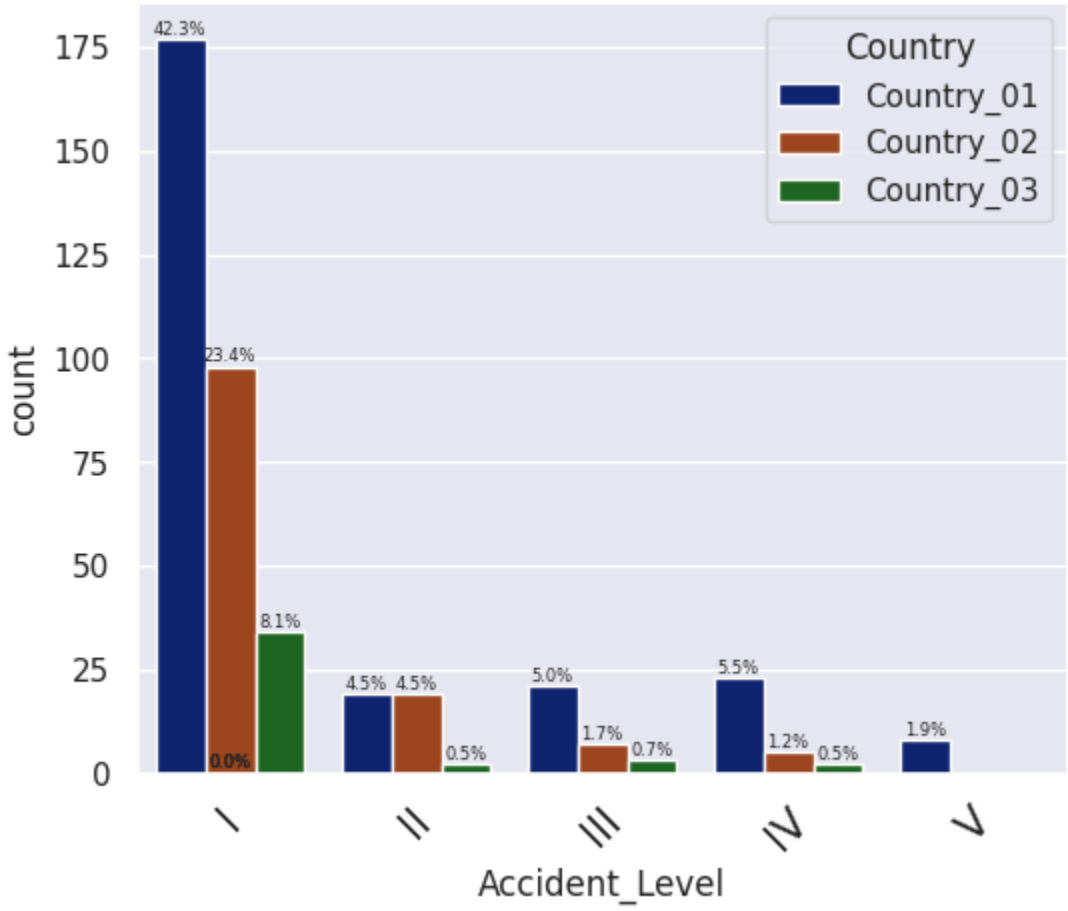


Observations:

- In Country 1, most accidents occurred by Third Party employees: 33%, followed by Employees: 20.8% and 5.5% occurred by Third Party (Remote).
- In Country 2, Employee accidents were at 20.1%, Third Party (Remote) at 7.7% and only 3.1% for Third Party employees.
- In Country 3, there were no accidents by Third Party (Remote), 8.1% by Third Party and only 1.7% by employees.

Accidents per Country

Accident Level break down for each country:

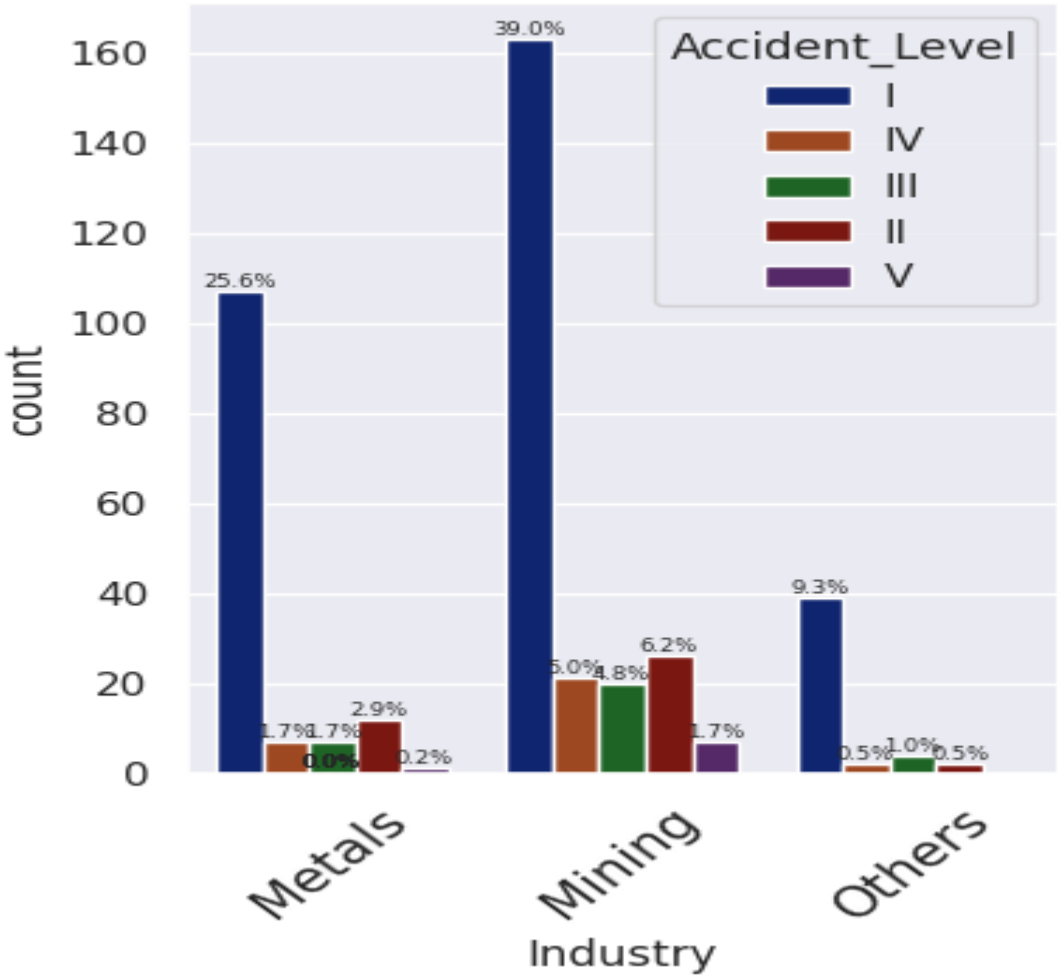


Observations:

- Accident Level 1 has the highest contribution across all countries.
- Accident Level 2 is consistent in Country 1 and 2 but less in country 3.
- Accident Level 3 and 4 are more in Country 1 than 2 and 3.
- Accident Level 5 is only present in Country 1.

Accident Types for various Industries

Accident Level break down for each industry:

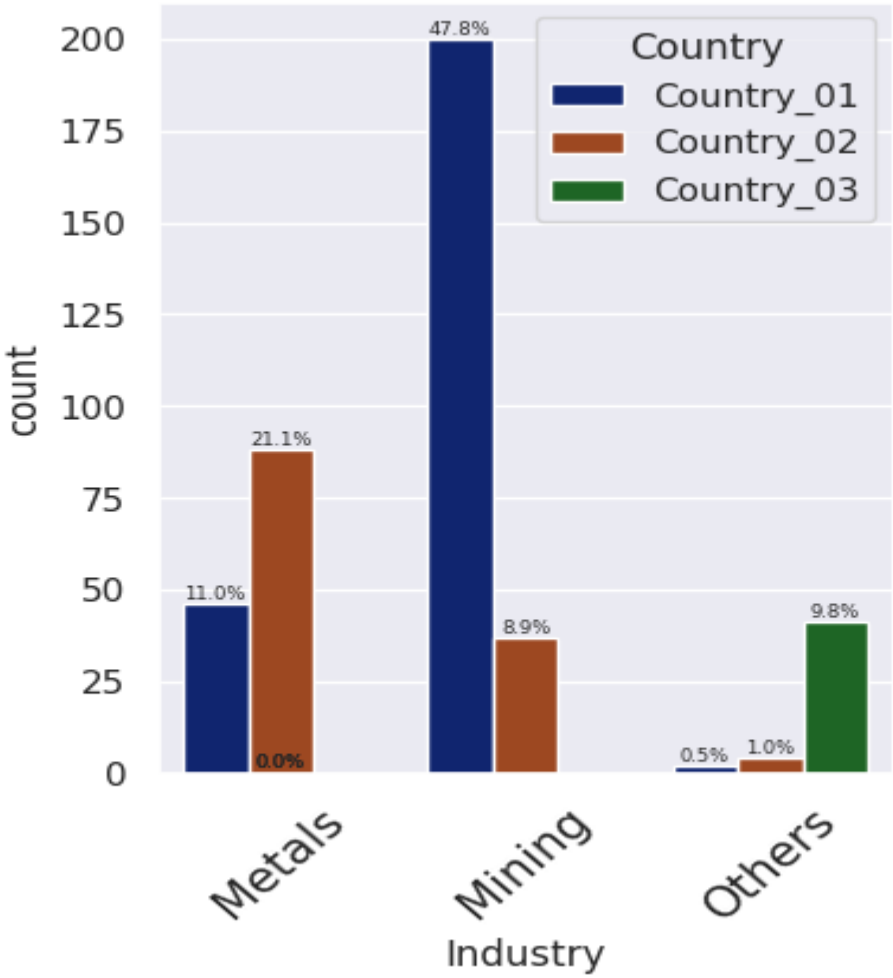


Observations:

- Accident Level 1 has the highest contribution across all industries.
- The Mining Industry has the highest count for all accident types.

Accidents in Countries based on Industry

Country break down for each industry:



Observations:

- Country 1 has 47.8% mining industry accidents out of the total number of accidents across countries and industries while 11% of total belongs to the metals industry for country 1.
- Country 2 has more accidents in metals than mining.
- Country 3 has accidents from other industries.

Model Building and Evaluation

Model Design

While building the model we need to take into consideration the observations from the EDA and the data which we have for building the model is limited. The most important factor in the data is the description which we have, and can be used to identify the Accidel Level which will be considered as the Target Variable.

To design the model we need to perform the preprocessing of the data:

1. Check duplicate rows and null, and take the necessary actions
2. Clean the text in the Description field
3. Divide the Date in Day, Month, Year

So we take 2 basic approaches into consideration for building the model.

1. Use Pre-trained model i.e. Transformers and use Classifiers on top of it.
2. Use Tokenizer and Glove (for creating vectors) and use Classifiers on top of it.

Models

The team decided to use both the approaches to build the model, and compare the models.

Model 1 - Tokenizer + GloVe

The model allows us to utilize the Description along with other fields which are available for us to derive the valuable insights from it. As we have used the Tokenizer + Glove, it helps us derive the vector embeddings from the description.

Additionally, we have derived the additional fields from the remaining existing fields using the dummies. This method gave the model to derive more insights to predict the Target variable which is the Accident Level.

Classifiers used for predicting the Target Variable:

1. Random Forest
2. Decision Tree
3. SVC
4. XGBoost

Also, we added the SMOTE to balance the data and then run the classifiers again on the balanced data.

Model 2 - Transformers

The first model was built on the Transformers, where the SentenseTransformer was used to create the embedding for the description.

This model is looking at only the description to predict the Target variable, using the classifier. The approach very well restricts the use of other parameters and to gain insights from it.

Classifiers used for predicting the Target Variable:

1. Random Forest
2. Decision Tree
3. SVC
4. XG Boost

Advance ML Models like DL and Transformers used for predicting the Target Variable:

1. Neural Network (DL) Algorithm
2. BiLSTM with Glove + Tokenizer
3. BiLSTM with SentenceTransformer

Benchmarking - Comparison of Experiments

Milestone 1

Model Part 1 - Traditional ML Algorithms

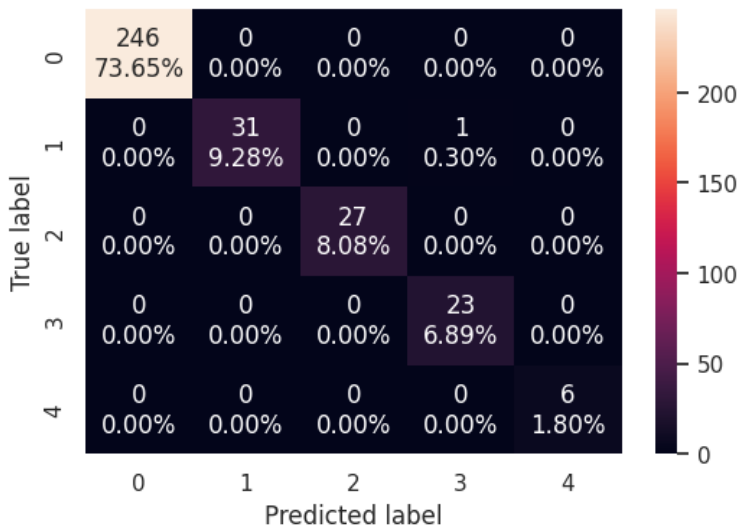
Model Part 1.1 Tokenizer + GloVe Without SMOTE

This model additionally uses the dummies to get additional features.

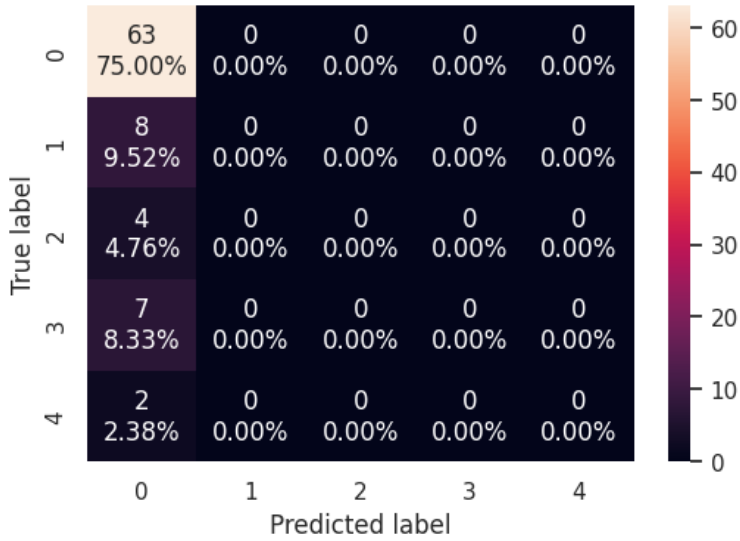
Random Forest Classifier

Random Forest does a good job and provides accuracy around 75%.

Confusion Matrix:Train Data



Confusion Matrix: Test Data



Classification Matrix: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	0.97	0.98	32
2	1.00	1.00	1.00	27
3	0.96	1.00	0.98	23
4	1.00	1.00	1.00	6
accuracy			1.00	334
macro avg	0.99	0.99	0.99	334
weighted avg	1.00	1.00	1.00	334

Classification Matrix: Test Data

```
              precision    recall  f1-score   support

           0       0.75      1.00      0.86        63
           1       0.00      0.00      0.00         8
           2       0.00      0.00      0.00         4
           3       0.00      0.00      0.00         7
           4       0.00      0.00      0.00         2

    accuracy                           0.75        84
   macro avg       0.15      0.20      0.17        84
weighted avg       0.56      0.75      0.64        84
```

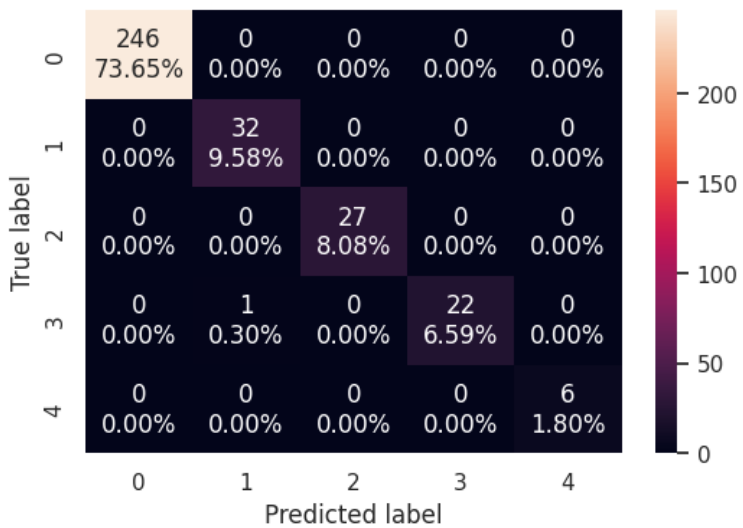
Observations:

- RandomForestClassifier Accuracy Train vs Test : 100% vs 75%
- RandomForestClassifier Precision Train vs Test : 100% vs 56%
- RandomForestClassifier Recall Train vs Test : 100% vs 75%
- RandomForestClassifier Model is not performing well since it is overfitting on Training Data

Decision Tree Classifier

The Decision Tree lacks to get better accuracy with vectors and the additional features providing accuracy of only 53%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data

Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	0.97	1.00	0.98	32
2	1.00	1.00	1.00	27
3	1.00	0.96	0.98	23
4	1.00	1.00	1.00	6
accuracy			1.00	334
macro avg	0.99	0.99	0.99	334
weighted avg	1.00	1.00	1.00	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.72	0.67	0.69	63
1	0.11	0.12	0.12	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.25	1.00	0.40	2
accuracy			0.54	84
macro avg	0.22	0.36	0.24	84
weighted avg	0.56	0.54	0.54	84

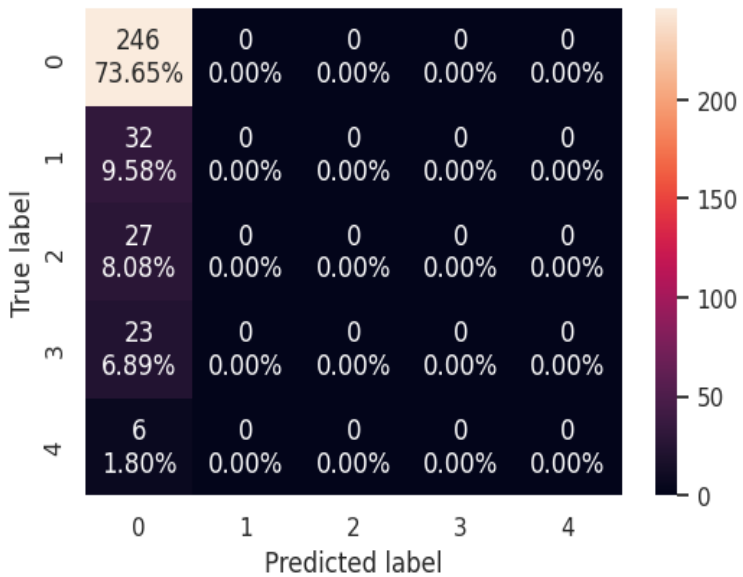
Observations:

- DecisionTreeClassifier Accuracy Train vs Test : 100% vs 54%
- DecisionTreeClassifier Precision Train vs Test : 100% vs 56%
- DecisionTreeClassifier Recall Train vs Test : 100% vs 54%
- The DecisionTreeClassifier Model is not performing well since it is overfitting on Training Data.
- RandomForestClassifier is performing little better than DecisionTreeClassifier

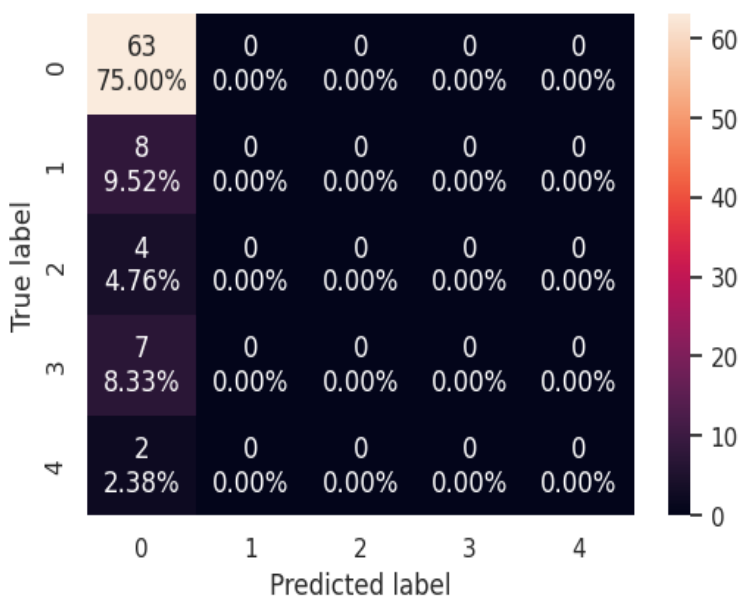
SVC

SVC provides a similar accuracy of 75% when combined with Transformer mode.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	0.74	1.00	0.85	246
1	0.00	0.00	0.00	32
2	0.00	0.00	0.00	27
3	0.00	0.00	0.00	23
4	0.00	0.00	0.00	6
accuracy			0.74	334
macro avg	0.15	0.20	0.17	334
weighted avg	0.54	0.74	0.62	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.75	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.56	0.75	0.64	84

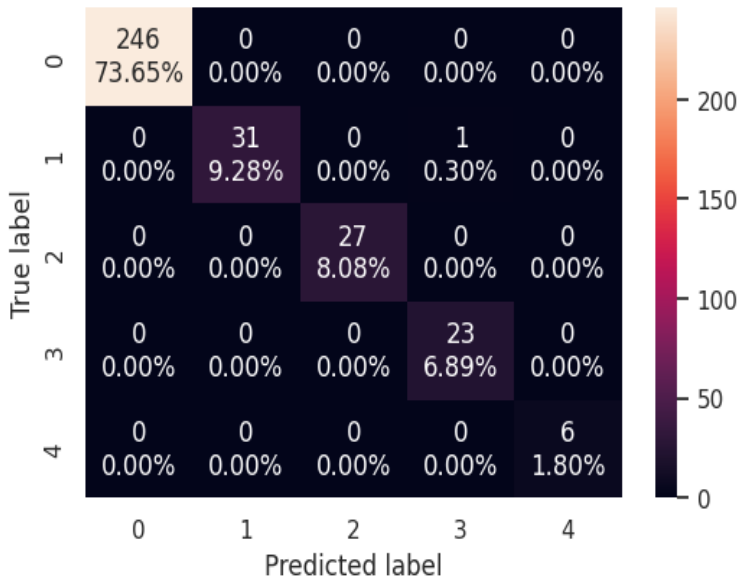
Observations:

- SVC_Classifier Accuracy Train vs Test : 74% vs 75%
- SVC_Classifier Precision Train vs Test : 54% vs 56%
- SVC_Classifier Recall Train vs Test : 74% vs 75%
- SVC_Classifier Model is performing well when compared with RandomForestClassifier and DecisionTreeClassifier

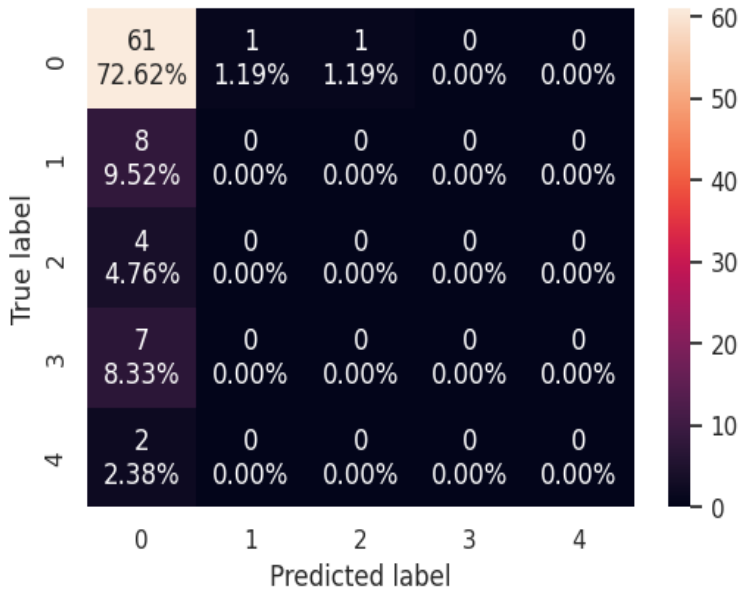
XG Boost

The XG Boot model loses the accuracy over the Transformer model and provides the accuracy of 74%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	0.97	0.98	32
2	1.00	1.00	1.00	27
3	0.96	1.00	0.98	23
4	1.00	1.00	1.00	6
accuracy			1.00	334
macro avg	0.99	0.99	0.99	334
weighted avg	1.00	1.00	1.00	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.74	0.97	0.84	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.73	84
macro avg	0.15	0.19	0.17	84
weighted avg	0.56	0.73	0.63	84

Observations:

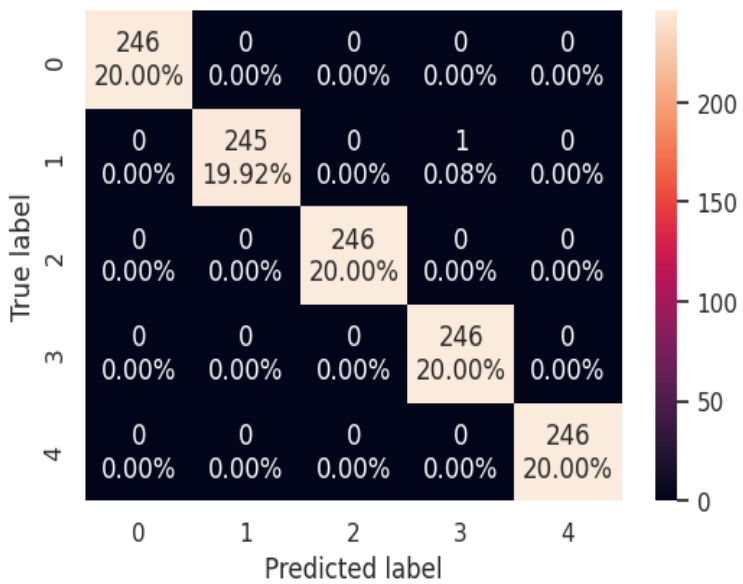
- XG_Boost Accuracy Train vs Test : 100% vs 73%
- XG_Boost Precision Train vs Test : 100% vs 56%
- XG_Boost Recall Train vs Test : 100% vs 73%
- XG_Boost Model is not performing well since it is overfitting on Training Data.
- XG_Boost is performing better than DecisionTreeClassifier and RandomForestClassifier
- SVC_Classifier is performing better than all other algorithms

Model Part 1.2 Tokenizer + GloVe WITH SMOTE

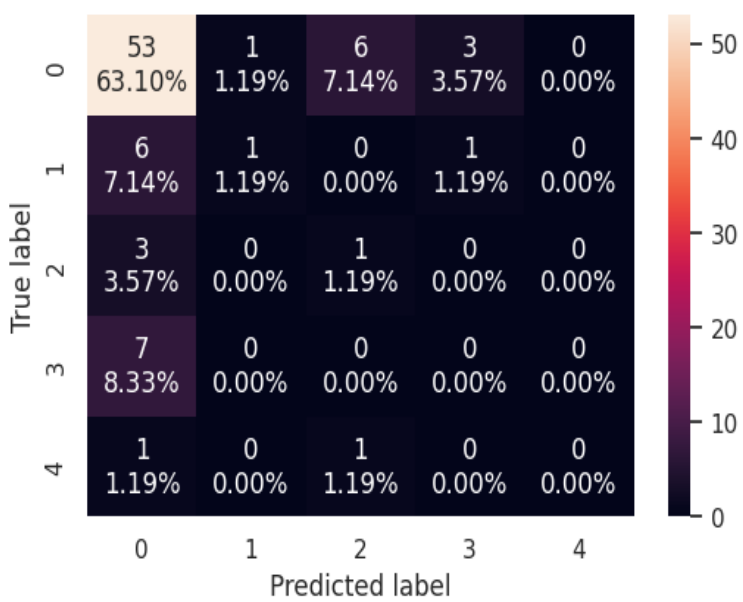
Random Forest Classifier

Random Forest does accuracy drops with SMOTE and provides accuracy around 64%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Matrix: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Matrix: Test Data

	precision	recall	f1-score	support
0	0.76	0.84	0.80	63
1	0.50	0.12	0.20	8
2	0.12	0.25	0.17	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.65	84
macro avg	0.28	0.24	0.23	84
weighted avg	0.62	0.65	0.62	84

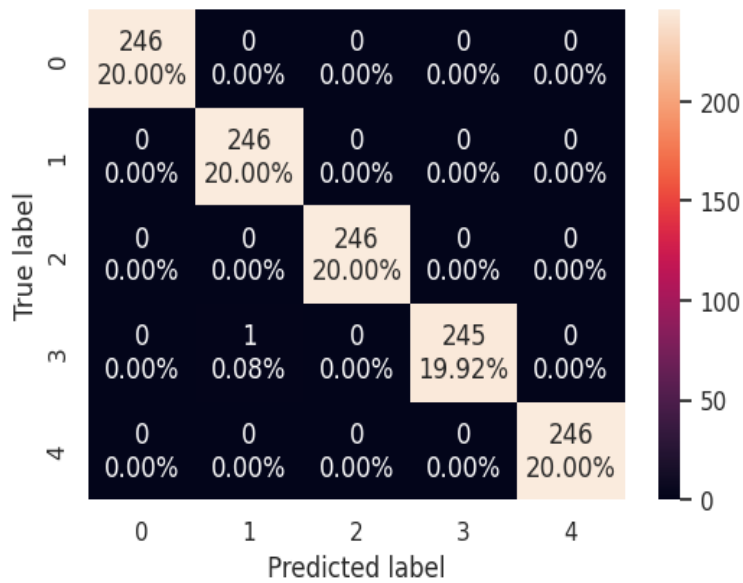
Observations:

- RandomForestClassifier + SMOTE Accuracy Train vs Test : 100% vs 65%
- RandomForestClassifier + SMOTE Precision Train vs Test : 100% vs 62%
- RandomForestClassifier + SMOTE Recall Train vs Test : 100% vs 65%
- RandomForestClassifier + SMOTE Model is not performing well since it is overfitting on Training Data
- RandomForestClassifier is performing better than RandomForestClassifier + SMOTE

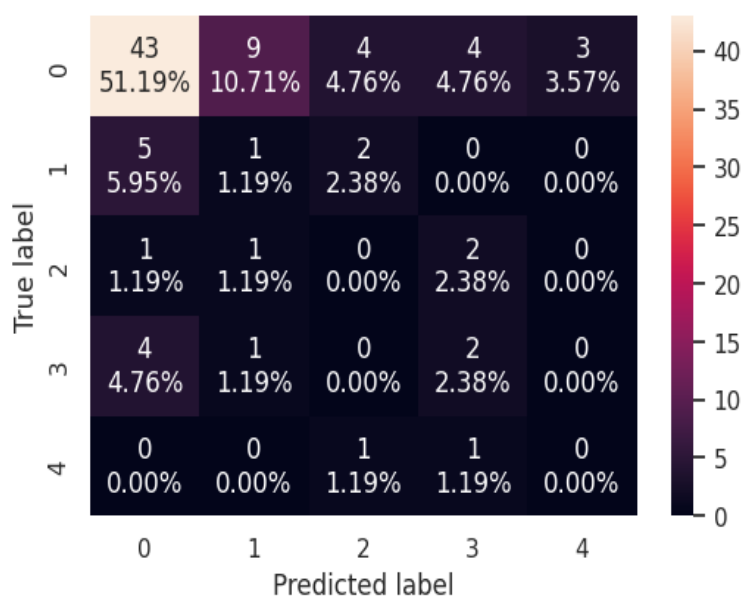
Decision Tree Classifier

The model performance drops further with SMOTE and provides only 45%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.81	0.68	0.74	63
1	0.08	0.12	0.10	8
2	0.00	0.00	0.00	4
3	0.22	0.29	0.25	7
4	0.00	0.00	0.00	2
accuracy			0.55	84
macro avg	0.22	0.22	0.22	84
weighted avg	0.63	0.55	0.59	84

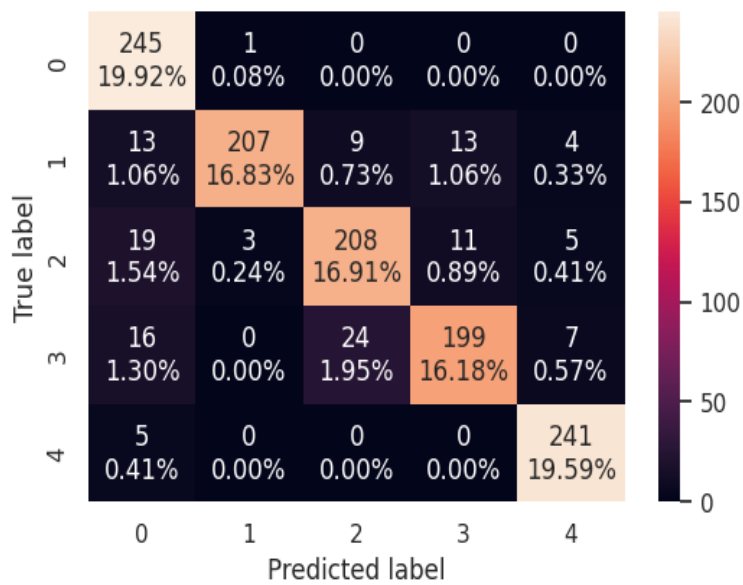
Observations:

- DecisionTreeClassifier + SMOTE Accuracy Train vs Test : 100% vs 55%
- DecisionTreeClassifier + SMOTE Precision Train vs Test : 100% vs 63%
- DecisionTreeClassifier + SMOTE Recall Train vs Test : 100% vs 55%
- DecisionTreeClassifier + SMOTE Model is not performing well since it is overfitting on Training Data.
- DecisionTreeClassifier + SMOTE is performing little better than DecisionTreeClassifier
- Both DecisionTreeClassifier and DecisionTreeClassifier + SMOTE not performing well when compared to RandomForestClassifier

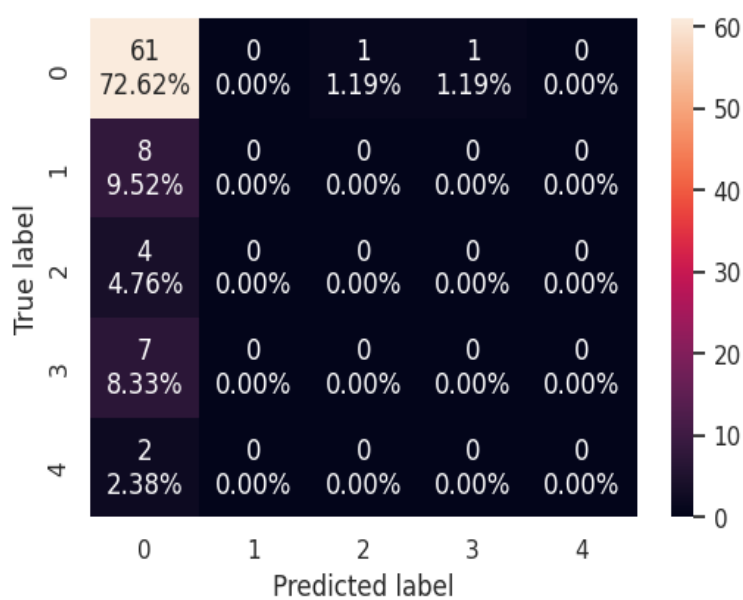
SVC

The Model performance drops further with SMOTE and provides the accuracy of 73%. It seems the SVC models give similar performance with all combinations.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	0.82	1.00	0.90	246
1	0.98	0.84	0.91	246
2	0.86	0.85	0.85	246
3	0.89	0.81	0.85	246
4	0.94	0.98	0.96	246
accuracy			0.89	1230
macro avg	0.90	0.89	0.89	1230
weighted avg	0.90	0.89	0.89	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.74	0.97	0.84	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.73	84
macro avg	0.15	0.19	0.17	84
weighted avg	0.56	0.73	0.63	84

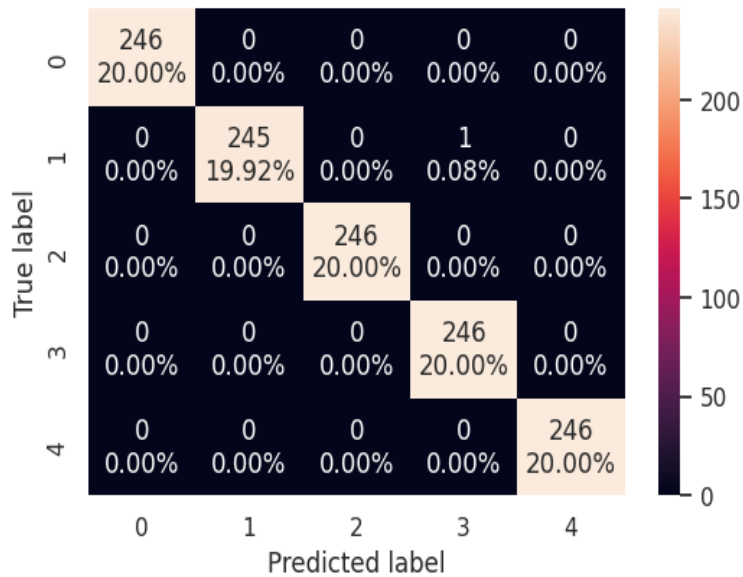
Observations:

- SVC_Classifier + SMOTE Accuracy Train vs Test : 89% vs 73%
- SVC_Classifier + SMOTE Precision Train vs Test : 90% vs 56%
- SVC_Classifier + SMOTE Recall Train vs Test : 89% vs 73%
- SVC_Classifier is performing little better than SVC_Classifier + SMOTE
- SVC_Classifier is performing better than all other models because it is not overfitting on Training and has performing consistent on Test data

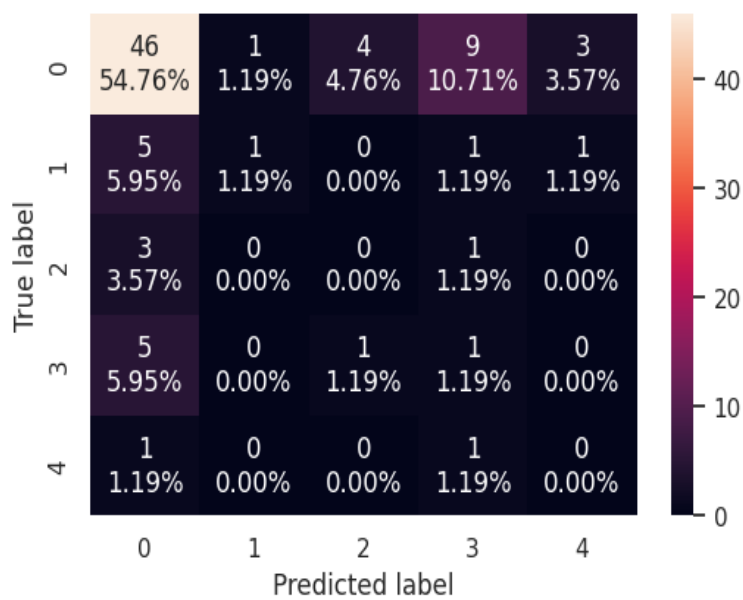
XG Boost

The model gives a performance of 64% with SMOTE.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.77	0.73	0.75	63
1	0.50	0.12	0.20	8
2	0.00	0.00	0.00	4
3	0.08	0.14	0.10	7
4	0.00	0.00	0.00	2
accuracy			0.57	84
macro avg	0.27	0.20	0.21	84
weighted avg	0.63	0.57	0.59	84

Observations:

- (XG_Boost + SMOTE) Accuracy Train vs Test : 100% vs 57%
- (XG_Boost + SMOTE) Precision Train vs Test : 100% vs 63%
- (XG_Boost + SMOTE) Recall Train vs Test : 100% vs 57%
- (XG_Boost + SMOTE) Model is not performing well since it is overfitting on Training Data.
- XG_Boost is performing better than (XG_Boost + SMOTE)

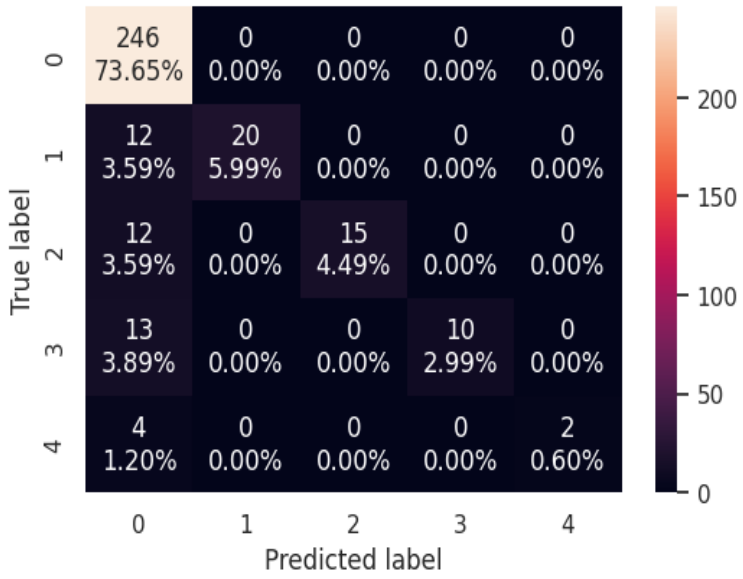
Model Part 1.3 Transformers

The Sentence Transformers have been used to evaluate the Descriptions, which gives the base to the Classifiers.

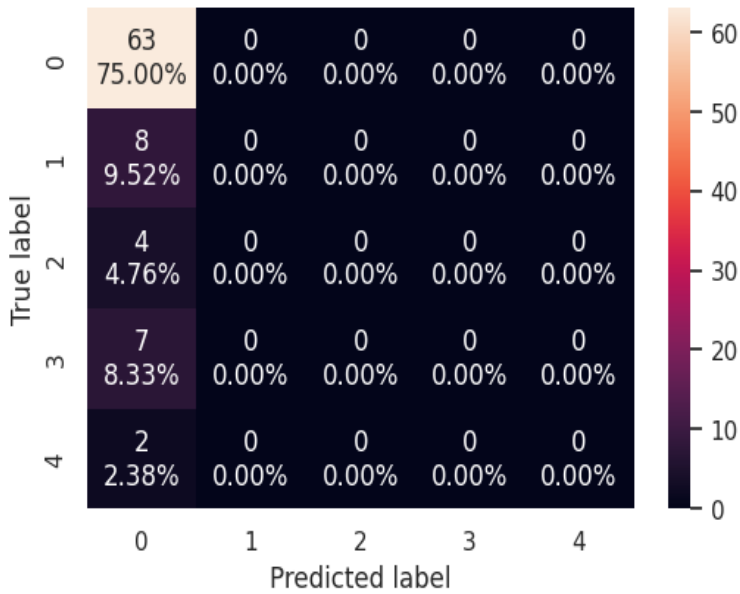
Random Forest Classifier

Random Forest does a good job and provides accuracy around 75%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	0.86	1.00	0.92	246
1	1.00	0.62	0.77	32
2	1.00	0.56	0.71	27
3	1.00	0.43	0.61	23
4	1.00	0.33	0.50	6
accuracy			0.88	334
macro avg	0.97	0.59	0.70	334
weighted avg	0.89	0.88	0.86	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.75	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.56	0.75	0.64	84

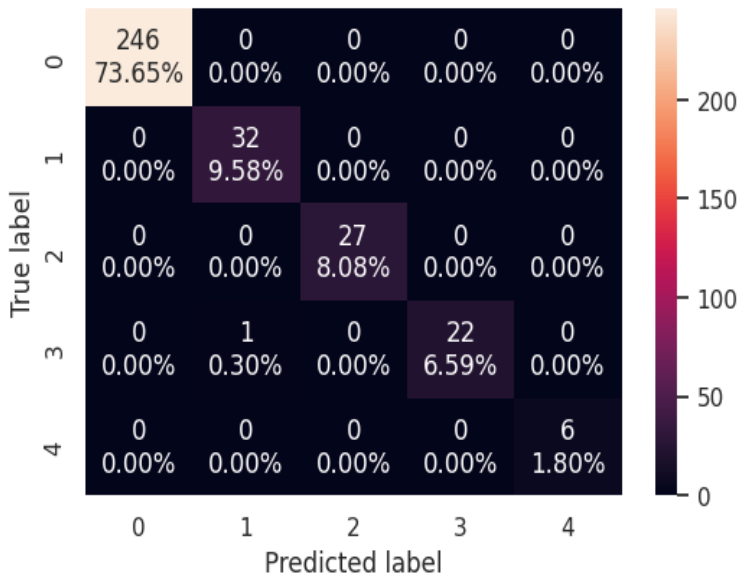
Observations:

- RandomForestClassifier + Transformers Accuracy Train vs Test : 89% vs 75%
- RandomForestClassifier + Transformers Precision Train vs Test : 88% vs 56%
- RandomForestClassifier + Transformers Recall Train vs Test : 89% vs 75%
- RandomForestClassifier + Transformers Model is not overfitting on Training Data and performing better than RandomForestClassifier

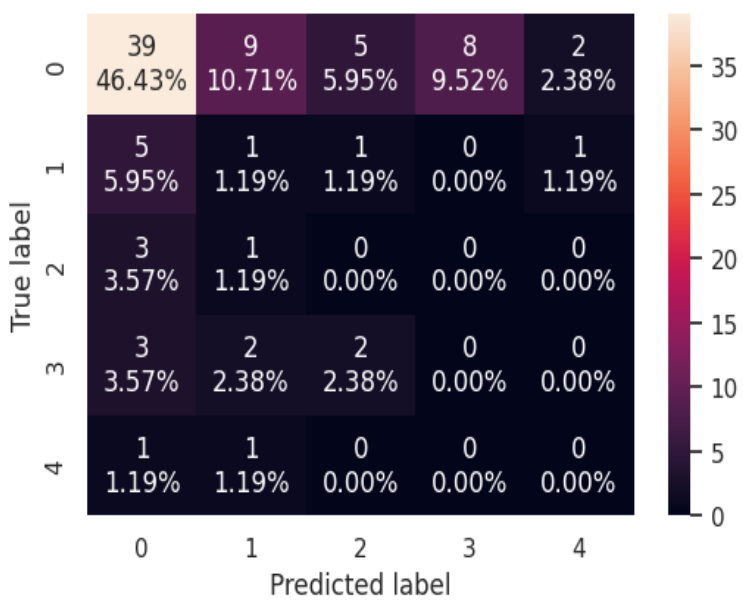
Decision Tree Classifier

The Decision Tree Model lacks a little bit and provides accuracy around only 53%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	0.97	1.00	0.98	32
2	1.00	1.00	1.00	27
3	1.00	0.96	0.98	23
4	1.00	1.00	1.00	6
accuracy			1.00	334
macro avg	0.99	0.99	0.99	334
weighted avg	1.00	1.00	1.00	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.76	0.62	0.68	63
1	0.07	0.12	0.09	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.48	84
macro avg	0.17	0.15	0.16	84
weighted avg	0.58	0.48	0.52	84

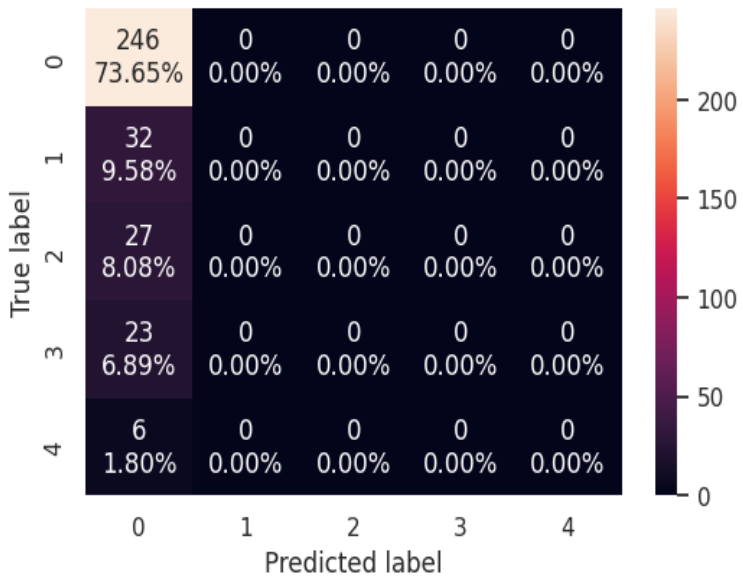
Observations:

- DecisionTreeClassifier + Transformers Accuracy Train vs Test : 100% vs 48%
- DecisionTreeClassifier + Transformers Precision Train vs Test : 100% vs 58%
- DecisionTreeClassifier + Transformers Recall Train vs Test : 100% vs 48%
- DecisionTreeClassifier + Transformers Model is not performing well since it is overfitting on Training Data.
- DecisionTreeClassifier + SMOTE is performing little better than DecisionTreeClassifier and DecisionTreeClassifier + Transformers

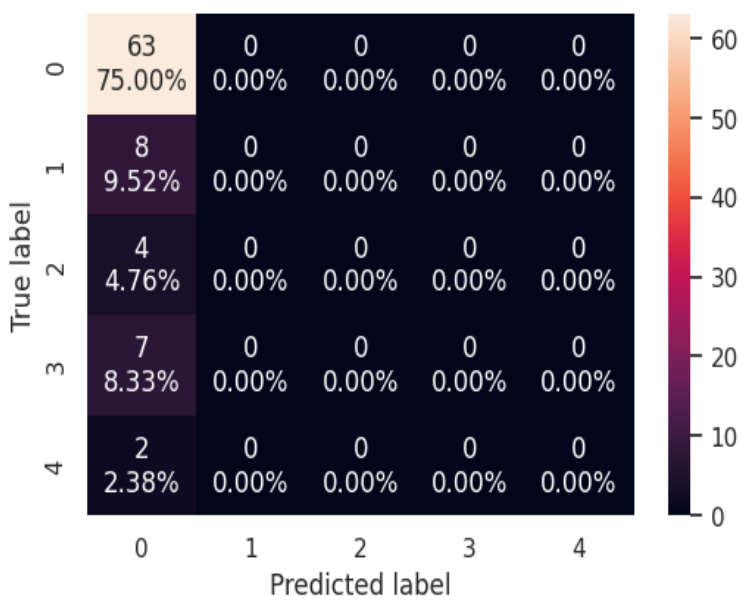
SVC

SVC Model performs similarly to the Random Forest with accuracy of 75%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	0.74	1.00	0.85	246
1	0.00	0.00	0.00	32
2	0.00	0.00	0.00	27
3	0.00	0.00	0.00	23
4	0.00	0.00	0.00	6
accuracy			0.74	334
macro avg	0.15	0.20	0.17	334
weighted avg	0.54	0.74	0.62	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.75	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.56	0.75	0.64	84

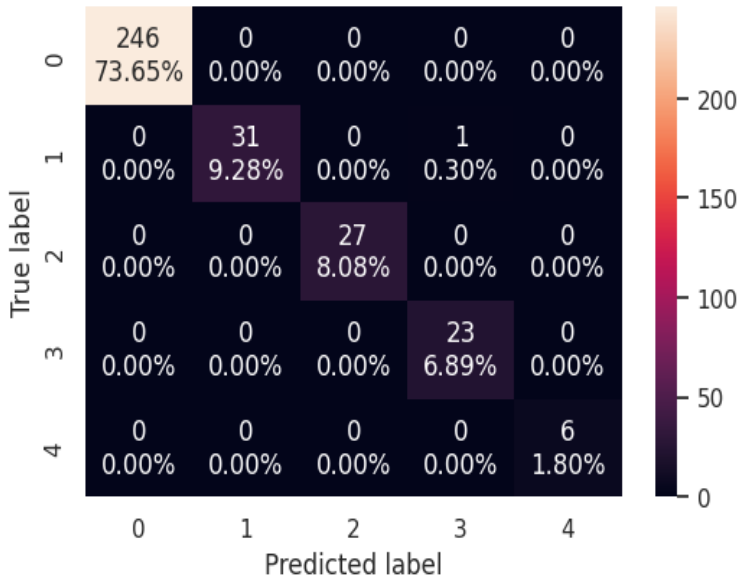
Observations:

- (SVC + Transformers) Accuracy Train vs Test : 74% vs 75%
- (SVC + Transformers) Precision Train vs Test : 54% vs 56%
- (SVC + Transformers) Recall Train vs Test : 74% vs 75%
- (SVC + Transformers) Model is performing well since it is not overfitting on Training Data and consistent on Test Data
- SVC and (SVC + Transformers) is performing better than (SVC + SMOTE)
- SVC and (SVC + Transformers) are performing same

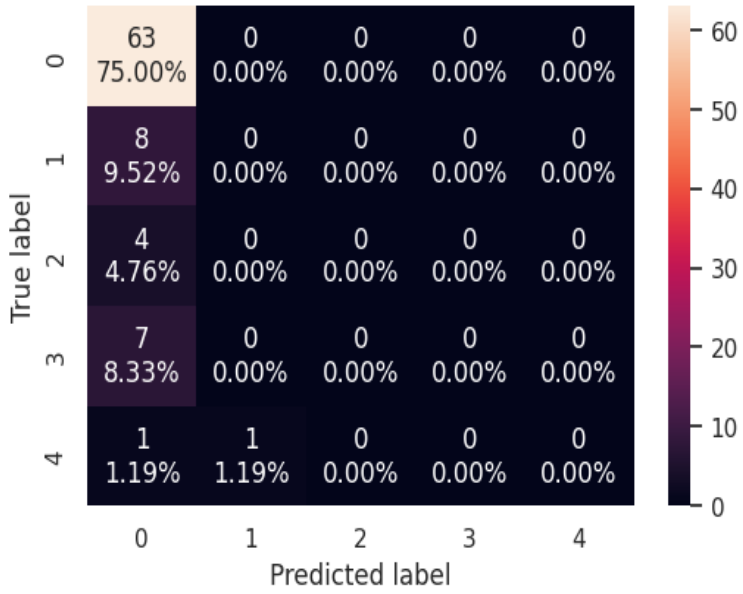
XG Boost

XG Boost almost meets the same results as Random Forest and SVC with accuracy of 74%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	0.97	0.98	32
2	1.00	1.00	1.00	27
3	0.96	1.00	0.98	23
4	1.00	1.00	1.00	6
accuracy			1.00	334
macro avg	0.99	0.99	0.99	334
weighted avg	1.00	1.00	1.00	334

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.76	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.57	0.75	0.65	84

Observations:

- (XG_Boost + Transformers) Accuracy Train vs Test : 100% vs 75%
- (XG_Boost + Transformers) Precision Train vs Test : 100% vs 57%
- (XG_Boost + Transformers) Recall Train vs Test : 100% vs 75%
- XG_Boost Model is not performing well since it is overfitting on Training Data.
- (XG_Boost + Transformers) is performing little better than XG_Boost and (XG_Boost + SMOTE)
- SVC and (SVC + Transformers) and (XG_Boost + Transformers) all are performing same

Models Summary

The models built have 2 main basic differentiation, one of which uses Transformers and other is built using Tokenizer and GloVe. The second model with Tokenizer and GloVe also uses the Dummies to create new features and SMOTE for balancing the data.

All the models perform almost similarly and we mostly get the accuracy between 60 to 75%. However, there is an exception with Decision Tree model along with Tokenizer + GloVe + SMOTE to underperform with only 45% accuracy followed by Transformers + Decision Tree model at 53%.

We can conclude the Decision Tree is not the best model to use in the scenario we are trying to explore for the HSE incidents.

Additionally, the Neural Network seems to overperform and provides the accuracy of 1 and loss of 0.1282, which suggests the overfitting of the model.

Milestone 2

Model Part 1.4 Hypertune ML Models

Hypertuning, or hyperparameter tuning, is the process of optimizing the hyperparameters of a machine learning model to improve its performance. Here are some key reasons why we hypertune:

1. Improve Model Performance: Properly tuned hyperparameters can significantly enhance the accuracy, precision, recall, and overall performance of the model.
2. Prevent Overfitting: By adjusting hyperparameters, we can control the complexity of the model, reducing the risk of overfitting to the training data and improving generalization to unseen data.
3. Optimize Training Time: Efficient hyperparameter settings can reduce the training time by avoiding unnecessarily complex models and ensuring that the model converges faster.
4. Balance Bias-Variance Trade-off: Hypertuning helps in finding the right balance between bias (error due to overly simplistic models) and variance (error due to overly complex models), leading to a more robust model.
5. Adapt to Specific Data: Different datasets have unique characteristics, and hypertuning allows the model to adapt to the specific patterns and distributions in the data, leading to better performance.

For the hypertuning phase, we utilize the df_merged dataframe, as detailed in Checkpoint3. This dataframe is generated following a comprehensive data cleansing process, machine learning preparation steps, and the application of the GloVe tokenizer on the Description column.

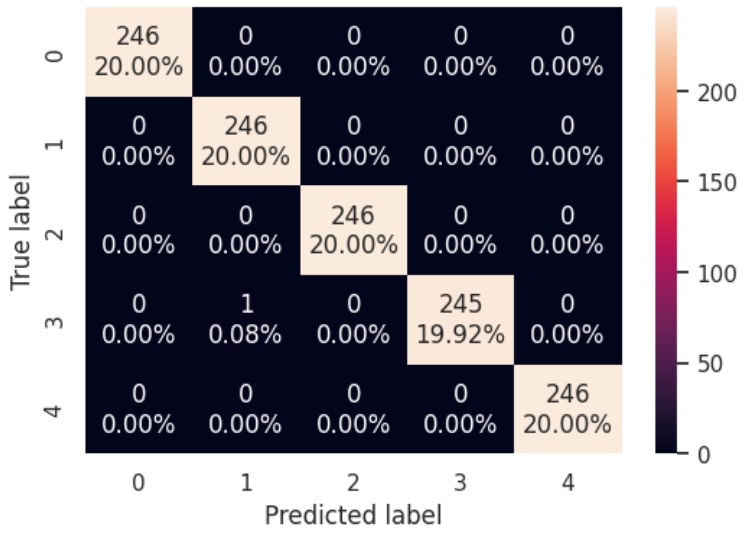
To address class imbalance, we apply SMOTE (Synthetic Minority Over-sampling Technique) exclusively on the training data within df_merged.

Subsequently, we split the data into training and testing sets, resulting in X_tr, X_te, y_tr, and y_te through the train_test_split function.

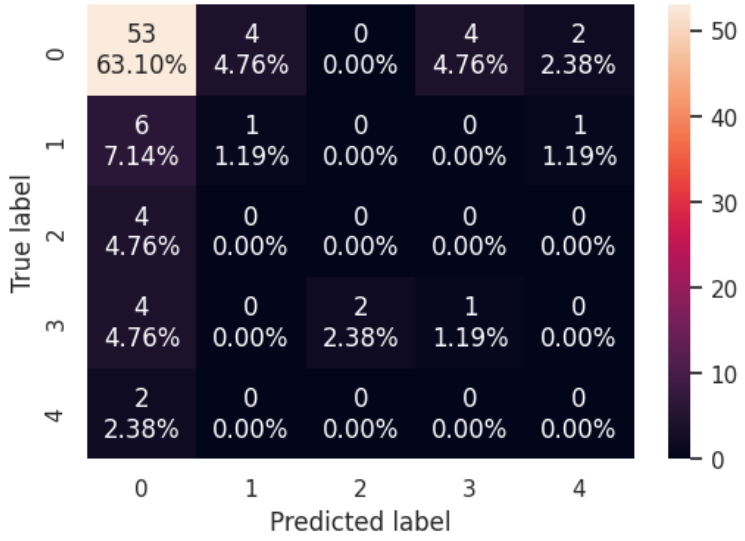
Hypertuned RandomForestClassifier

Random Forest does a good job and provides accuracy around 65%.

Confusion Matrix:Train Data



Confusion Matrix: Test Data



Classification Matrix: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Matrix: Test Data

	precision	recall	f1-score	support
0	0.77	0.84	0.80	63
1	0.20	0.12	0.15	8
2	0.00	0.00	0.00	4
3	0.20	0.14	0.17	7
4	0.00	0.00	0.00	2
accuracy			0.65	84
macro avg	0.23	0.22	0.22	84
weighted avg	0.61	0.65	0.63	84

Observations:

Stats:

- RandomForestClassifier Accuracy Train vs Test : 100% vs 65%
- RandomForestClassifier Precision Train vs Test : 100% vs 61%
- RandomForestClassifier Recall Train vs Test : 100% vs 65%
- RandomForestClassifier Model is not performing well since it is overfitting on Training Data

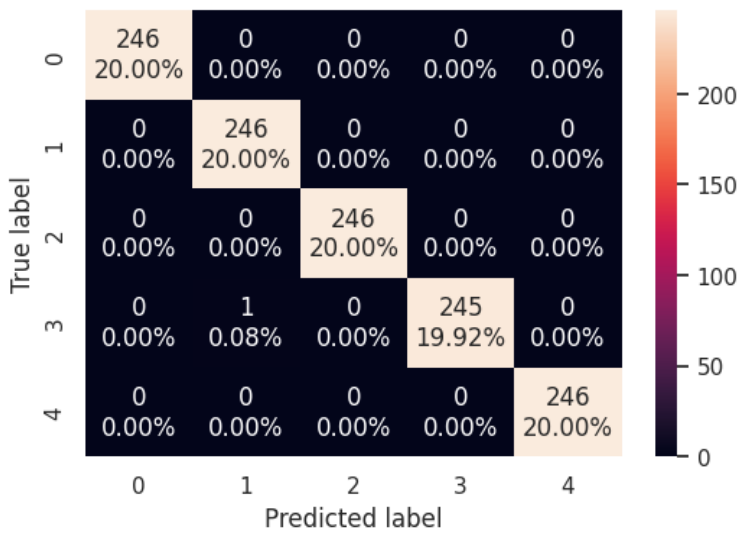
Highlights:

- The hypertuned random forest classifier shows excellent performance on the training data but struggles with generalization, particularly for minority classes in the test set.
- The stark contrast between the training and test performance suggests overfitting.
- The precision, recall, and f1-score for classes 1, 2, 3, and 4 are significantly lower compared to class 0 in the test set.
- The macro average for precision, recall, and f1-score on the test set is 0.25, 0.23, and 0.23, respectively, indicating poor performance across all classes.
- The weighted average is higher at 0.62, reflecting the model's better performance on the majority class (class 0).
- The best parameters found for the model are:
  Criterion: 'gini'
  Max Depth: 12
  Max Features: 15
  Number of Estimators: 100
- These parameters suggest a relatively deep tree with a substantial number of features considered at each split, which might contribute to the overfitting observed.
- Addressing overfitting and class imbalance could improve the model's performance on unseen data.

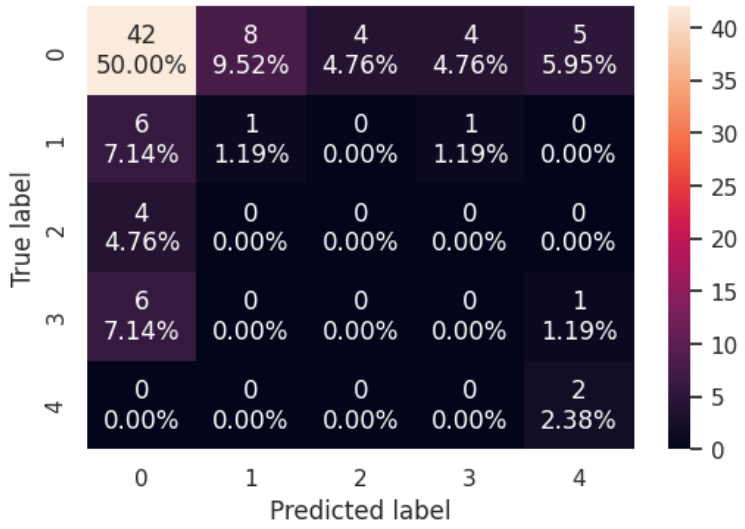
Hypertuned Decision Tree Classifier

The Decision Tree lacks to get better accuracy with vectors and the additional features providing accuracy of only 40%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.70	0.52	0.60	63
1	0.08	0.12	0.10	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.40	84
macro avg	0.16	0.13	0.14	84
weighted avg	0.53	0.40	0.46	84

Observations:

Stats:

- DecisionTreeClassifier Accuracy Train vs Test : 100% vs 40%
- DecisionTreeClassifier Precision Train vs Test : 100% vs 53%
- DecisionTreeClassifier Recall Train vs Test : 100% vs 40%
- The DecisionTreeClassifier Model is not performing well since it is overfitting on Training Data.
- RandomForestClassifier is performing little better than DecisionTreeClassifier

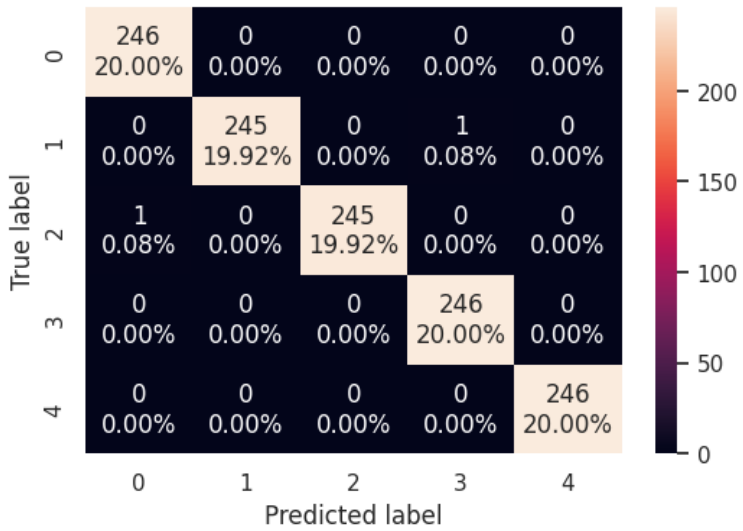
Highlights:

- Same as the previous model, the hypertuned DecisionTreeClassifier shows excellent performance on the training data but struggles with generalization, particularly for minority classes in the test set.
- The significant drop in performance from the training set to the test set suggests overfitting.
- While the training accuracy is 1.00, the test accuracy drops to 0.39, indicating that the model may not generalize well to unseen data.
- The macro average for precision, recall, and f1-score on the test set is 0.17, 0.15, and 0.15, respectively, indicating poor performance across all classes.
- The weighted average is higher at 0.55, 0.39, and 0.45, reflecting the model's better performance on the majority class (class 0).
- The best parameters found for the model are:
  Criterion: 'gini'
  Max Depth: 15
  Min Samples Leaf: 1
  Min Samples Split: 3
- These parameters suggest a relatively deep tree with minimal constraints on leaf and split samples, which might contribute to the overfitting observed.

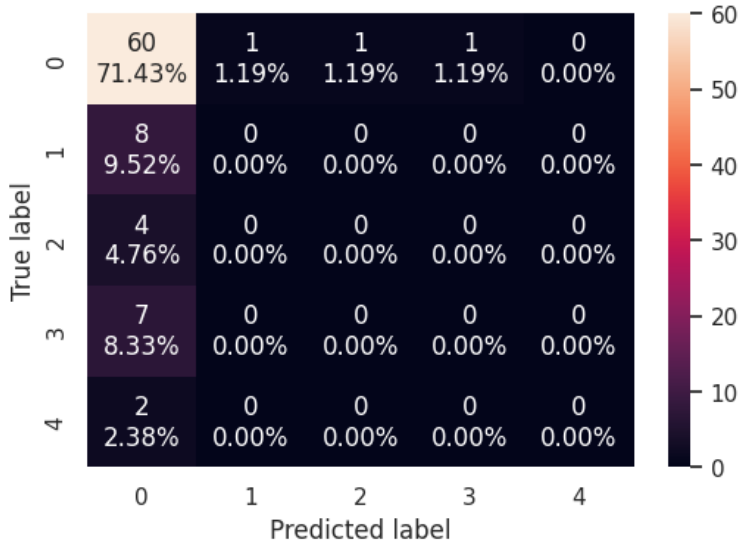
Hypertuned SVC

SVC provides a similar accuracy of 71% when combined with Transformer mode.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.74	0.95	0.83	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.71	84
macro avg	0.15	0.19	0.17	84
weighted avg	0.56	0.71	0.62	84

Observations:

Stats:

- SVC_Classifier Accuracy Train vs Test : 100% vs 71%
- SVC_Classifier Precision Train vs Test : 100% vs 56%
- SVC_Classifier Recall Train vs Test : 100% vs 71%
- SVC_Classifier Model is performing well when compared with RandomForestClassifier and DecisionTreeClassifier in test, but overfitting on Train data.

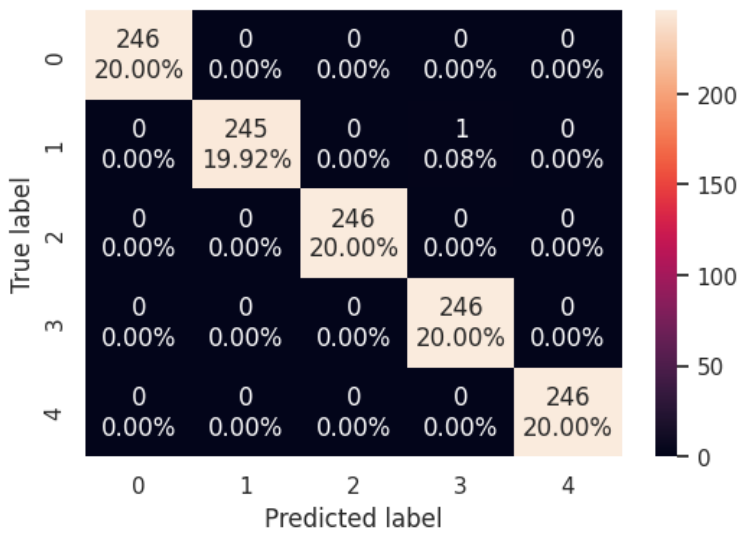
Highlights:

- Same as the previous models, the hypertuned SVC shows excellent performance on the training data but struggles with generalization, particularly for minority classes in the test set.
- While the training accuracy is 1.00, the test accuracy drops to 0.69, indicating overfitting. The model may not generalize well to unseen data.
- The macro average for precision, recall, and f1-score on the test set is 0.15, 0.18, and 0.16, respectively, indicating poor performance across all classes.
- The weighted average is higher at 0.55, 0.69, and 0.61, reflecting the model's better performance on the majority class (class 0).
- The best parameters found for the model are:
  C: 100
  Gamma: 0.1
  Kernel: 'rbf'
- These parameters suggest a strong regularization (high C value) and a relatively high gamma value, which might contribute to the overfitting observed.

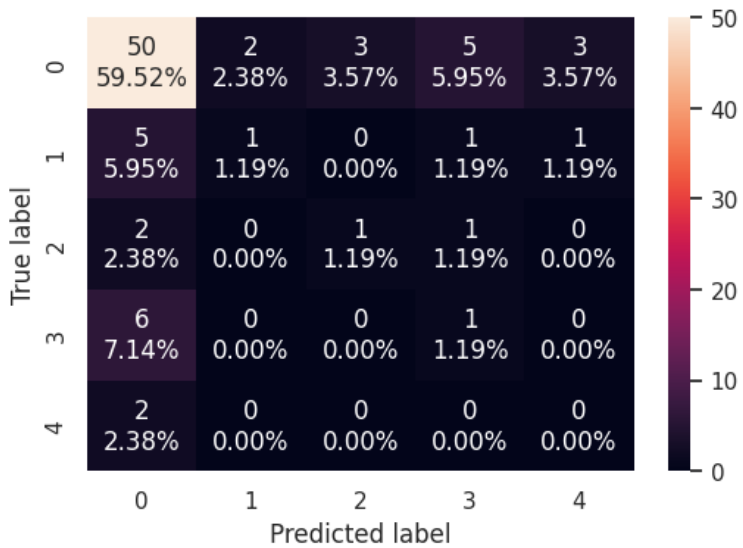
Hypertuned XG Boost

The XG Boot model loses the accuracy over the Transformer model and provides the accuracy of 63%.

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Report: Train Data

	precision	recall	f1-score	support
0	1.00	1.00	1.00	246
1	1.00	1.00	1.00	246
2	1.00	1.00	1.00	246
3	1.00	1.00	1.00	246
4	1.00	1.00	1.00	246
accuracy			1.00	1230
macro avg	1.00	1.00	1.00	1230
weighted avg	1.00	1.00	1.00	1230

Classification Report: Test Data

	precision	recall	f1-score	support
0	0.77	0.79	0.78	63
1	0.33	0.12	0.18	8
2	0.25	0.25	0.25	4
3	0.12	0.14	0.13	7
4	0.00	0.00	0.00	2
accuracy			0.63	84
macro avg	0.30	0.26	0.27	84
weighted avg	0.63	0.63	0.63	84

Observations:

Stats:

- XG_Boost Accuracy Train vs Test : 100% vs 63%
- XG_Boost Precision Train vs Test : 100% vs 63%
- XG_Boost Recall Train vs Test : 100% vs 63%
- XG_Boost Model is not performing well since it is overfitting on Training Data.
- XG_Boost is performing better than DecisionTreeClassifier and RandomForestClassifier

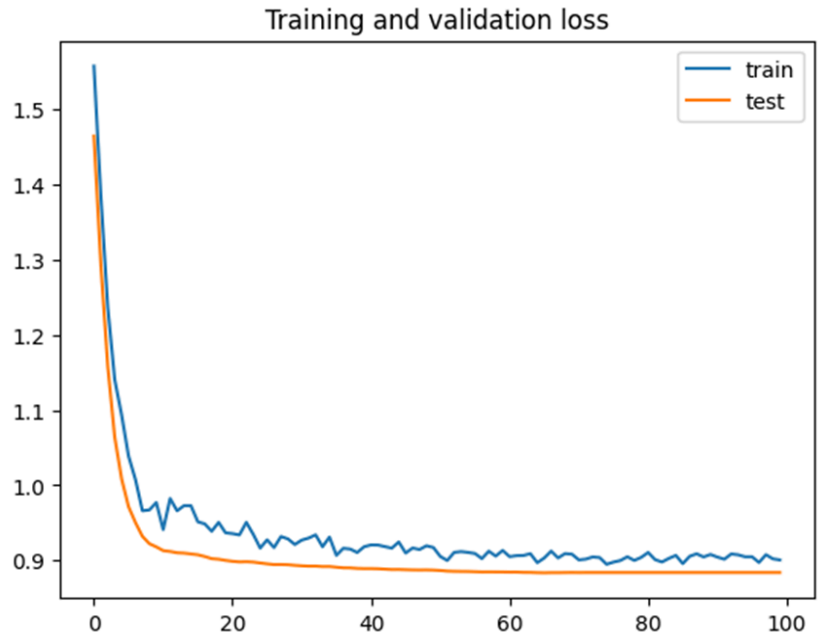
Highlights:

- Same as the previous models, the hypertuned XGBoost shows excellent performance on the training data but struggles with test set indicating overfitting.
- The precision, recall, and f1-score for classes 1, 2, 3, and 4 are significantly lower compared to class 0 in the test set.
- This suggests that the model struggles with minority classes.
- The macro average for precision, recall, and f1-score on the test set is 0.22, 0.18, and 0.19, respectively, indicating poor performance across all classes.
- The weighted average is higher at 0.60, 0.60, and 0.59, reflecting the model's better performance on the majority class (class 0).
- The best parameters found for the model are:
  Learning Rate: 0.1
  Max Depth: 7
  Subsample: 0.7
- These parameters suggest a moderately deep tree with a substantial portion of the data used for each tree, which might contribute to the overfitting observed.

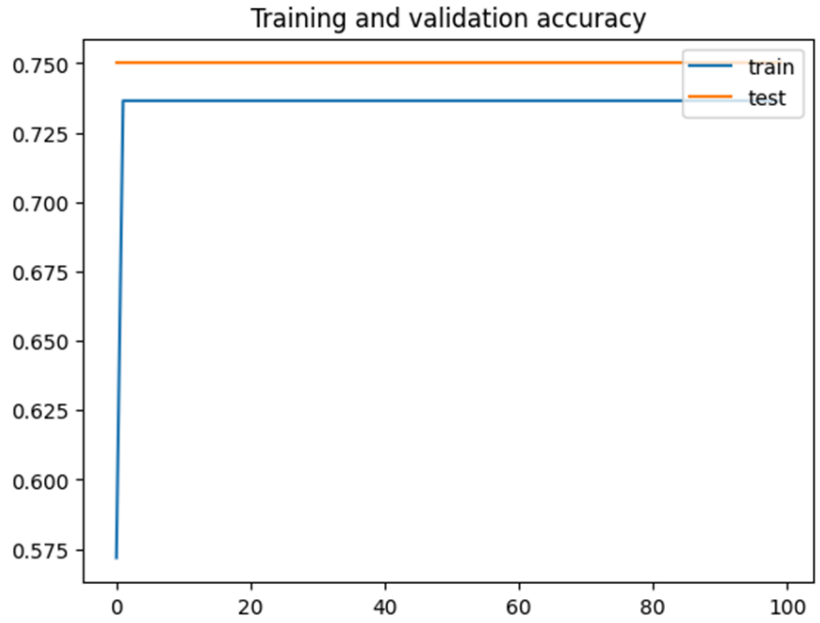
Model Part 2 - Advanced ML Models

Model Part 2.1 Neural Network Models

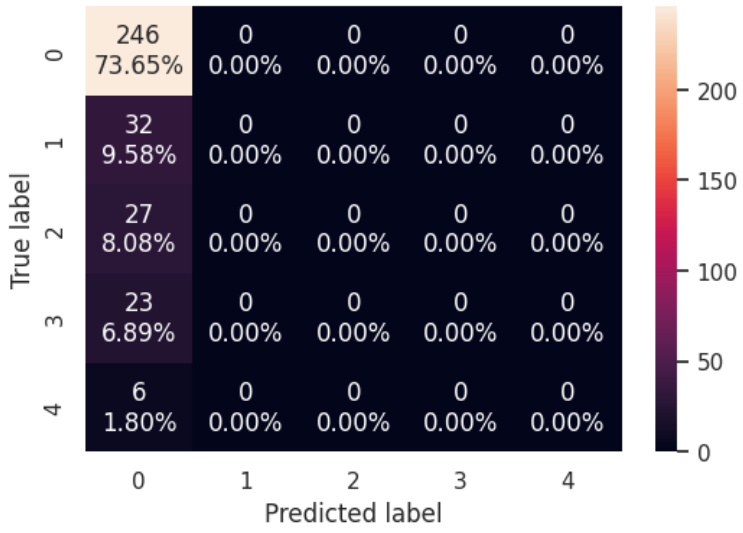
Validation Loss: Train & Test Data



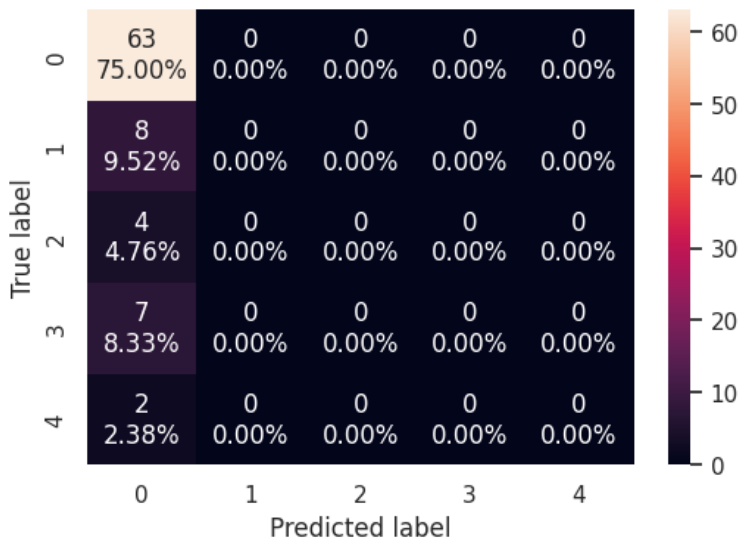
Accuracy: Train & Test Data



Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Matrix: Train Data

	precision	recall	f1-score	support
0	0.74	1.00	0.85	246
1	0.00	0.00	0.00	32
2	0.00	0.00	0.00	27
3	0.00	0.00	0.00	23
4	0.00	0.00	0.00	6
accuracy			0.74	334
macro avg	0.15	0.20	0.17	334
weighted avg	0.54	0.74	0.62	334

Classification Matrix: Test Data

	precision	recall	f1-score	support
0	0.75	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.56	0.75	0.64	84

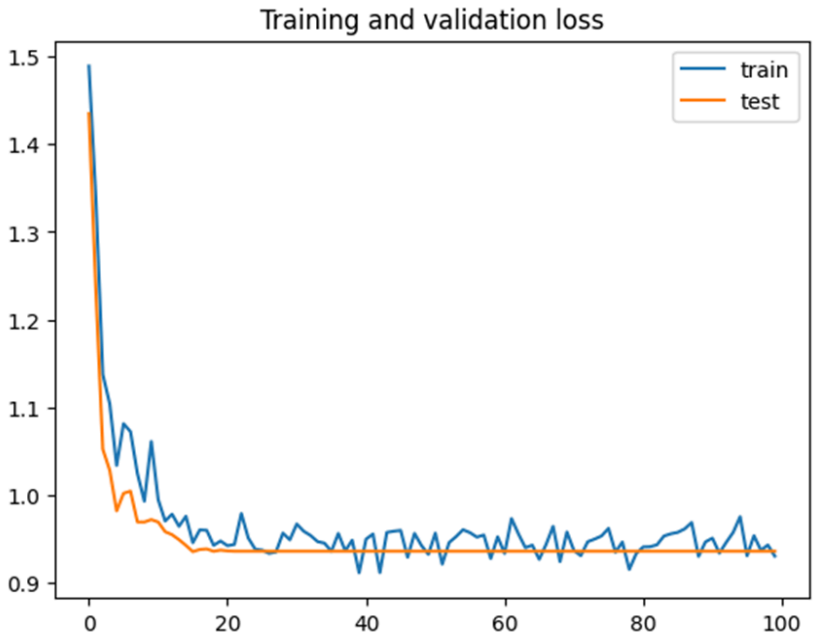
Observations:

1. **Consistent Performance Across Training and Testing:** The model shows similar performance metrics for both training and testing datasets, with accuracy, precision, and recall all around 0.74-0.75. This indicates that the model generalizes well and is not overfitting.
2. **Significant Improvement Over Epochs:** At the beginning of training (Epoch 1), the categorical accuracy was 0.4170 with a loss of 1.5942. By the end of training (Epoch 100), the categorical accuracy improved to 0.7458 with a reduced loss of 0.8748. This demonstrates that the model has learned effectively over the training period.
3. **Stable Validation Performance:** The validation categorical accuracy remained stable at 0.7500 from the first epoch to the last. This stability suggests that the

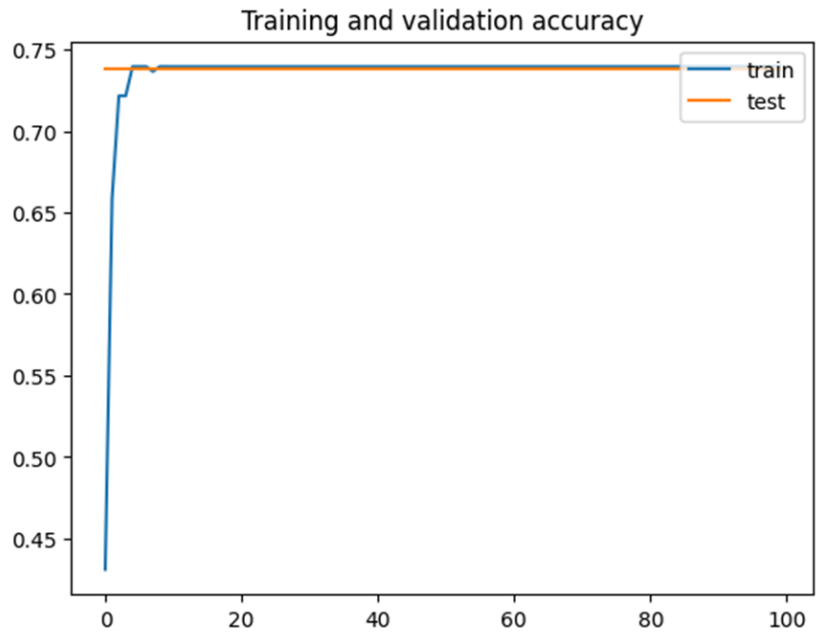
model's performance on unseen data is consistent, which is a positive sign for its reliability.

Model Part 2.2 BiLSTM (Glove + Tokenizer)

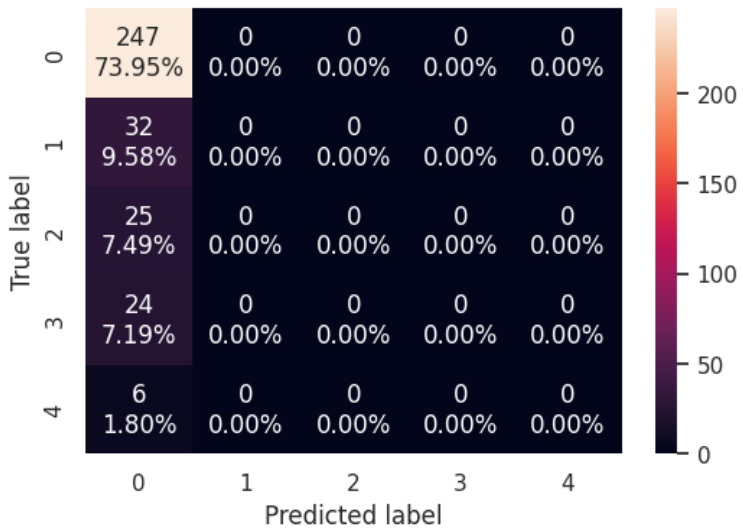
Validation Loss: Train & Test Data



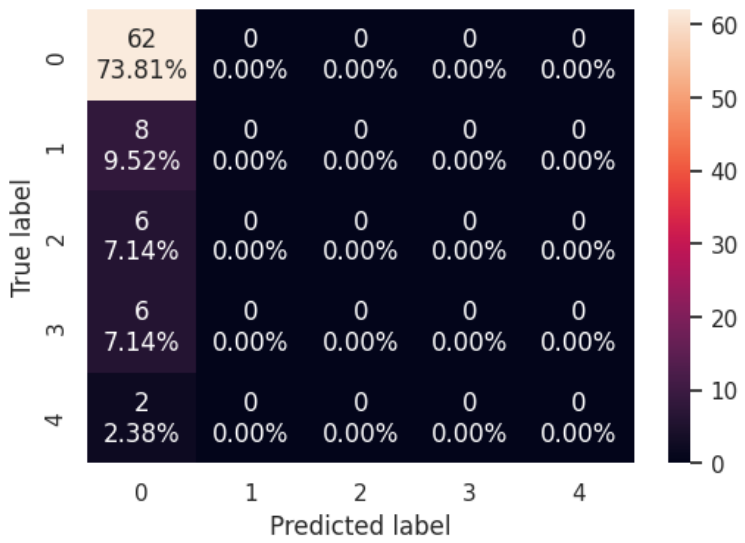
Accuracy: Train & Test Data



Confusion Matrix: Train Data



Confusion Matrix: Test Data


True label
Predicted label
0
1
2
3
4
62
73.81%
8
9.52%
6
7.14%
6
7.14%
2
2.38%
0
0.00%
60
50
40
30
20
10

Classification Matrix: Train Data

	precision	recall	f1-score	support
0	0.74	1.00	0.85	247
1	0.00	0.00	0.00	32
2	0.00	0.00	0.00	25
3	0.00	0.00	0.00	24
4	0.00	0.00	0.00	6
accuracy			0.74	334
macro avg	0.15	0.20	0.17	334
weighted avg	0.55	0.74	0.63	334

Classification Matrix: Test Data

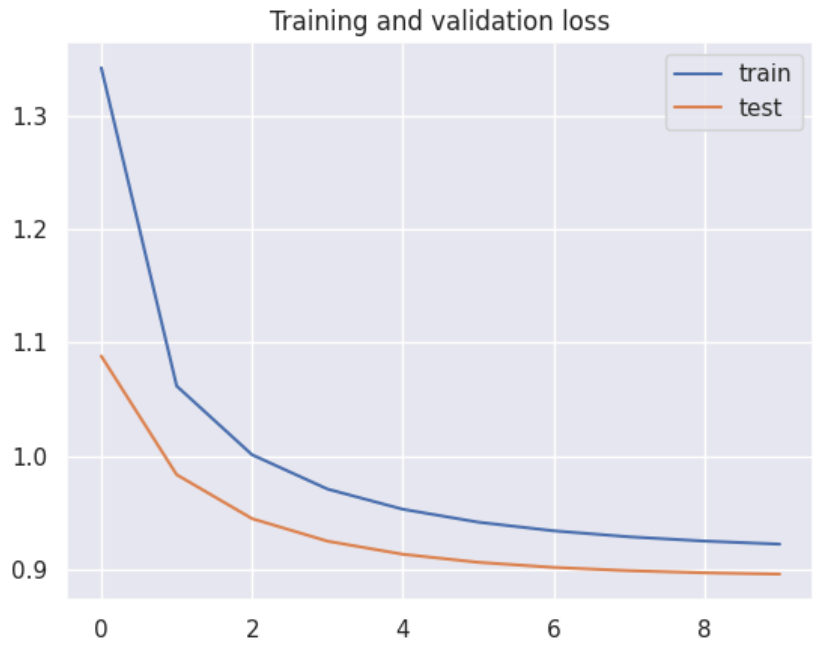
	precision	recall	f1-score	support
0	0.74	1.00	0.85	62
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	6
3	0.00	0.00	0.00	6
4	0.00	0.00	0.00	2
accuracy			0.74	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.54	0.74	0.63	84

Observations:

1. **Significant Improvement in Training Accuracy:** The training accuracy improved from 0.3143 in the first epoch to 0.7213 by the 100th epoch. This indicates that the model has effectively learned from the training data over time.
2. **Stable Validation Accuracy:** The validation accuracy remained stable at 0.7381 from the first epoch to the last. This suggests that the model's performance on unseen data is consistent, which is a positive sign for its generalization capabilities.
3. **Balanced Performance Metrics:** The training and testing accuracy, precision, and recall are all very close to each other (around 0.7395 for training and 0.7381 for testing). This indicates that the model is not overfitting and performs similarly on both training and testing datasets.

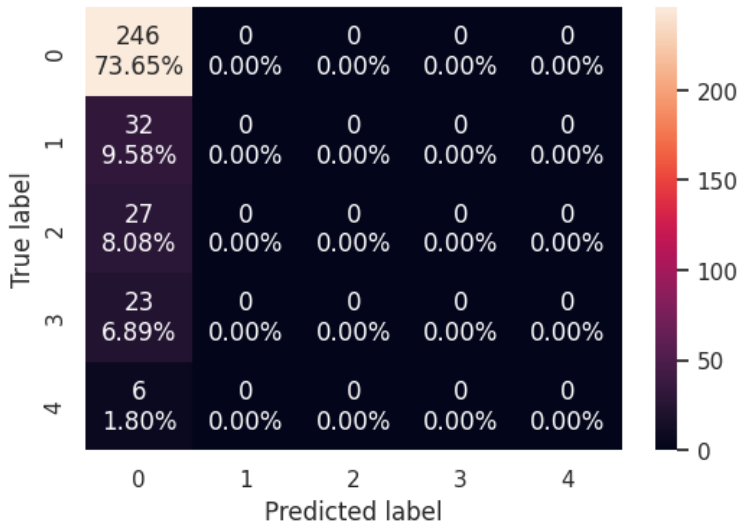
Model part 2.3 BiLSTM (Sentence Transformer)

Validation Loss: Train & Test Data

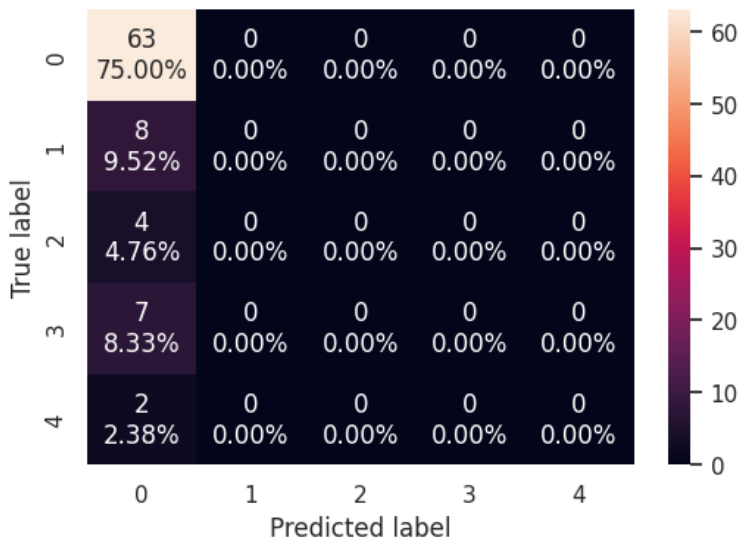


Accuracy: Train & Test Data

Confusion Matrix: Train Data



Confusion Matrix: Test Data



Classification Matrix: Train Data

	precision	recall	f1-score	support
0	0.74	1.00	0.85	246
1	0.00	0.00	0.00	32
2	0.00	0.00	0.00	27
3	0.00	0.00	0.00	23
4	0.00	0.00	0.00	6
accuracy			0.74	334
macro avg	0.15	0.20	0.17	334
weighted avg	0.54	0.74	0.62	334

Classification Matrix: Test Data

	precision	recall	f1-score	support
0	0.75	1.00	0.86	63
1	0.00	0.00	0.00	8
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	2
accuracy			0.75	84
macro avg	0.15	0.20	0.17	84
weighted avg	0.56	0.75	0.64	84

Observations:

1. **Consistent Validation Accuracy:** The validation accuracy is consistently at 0.7500, indicating that the model is performing steadily on the validation set.
2. **Improvement in Training Metrics:** There is a noticeable improvement in the training accuracy, precision, and recall from 0.739521 to 0.750000. This suggests that the model is learning and improving its performance on the training data.
3. **Loss Values:** The training loss is 0.8534, while the validation loss is slightly higher at 0.8830. This indicates that the model is generalizing well but still has room for improvement to reduce the validation loss further.

Models Summary

We evaluated 7 different models in this milestone, including hypertuning ML Models as well as Advanced Networks like “BiLSTM with Glove + Tokenizer Embeddings,” and “Neural Network.”

The models are evaluated based on three metrics: accuracy, precision, and recall.

Models like “Neural Network” and “BiLSTM with Sentence Transformer Embeddings” demonstrate balanced performance across accuracy, precision, and recall.

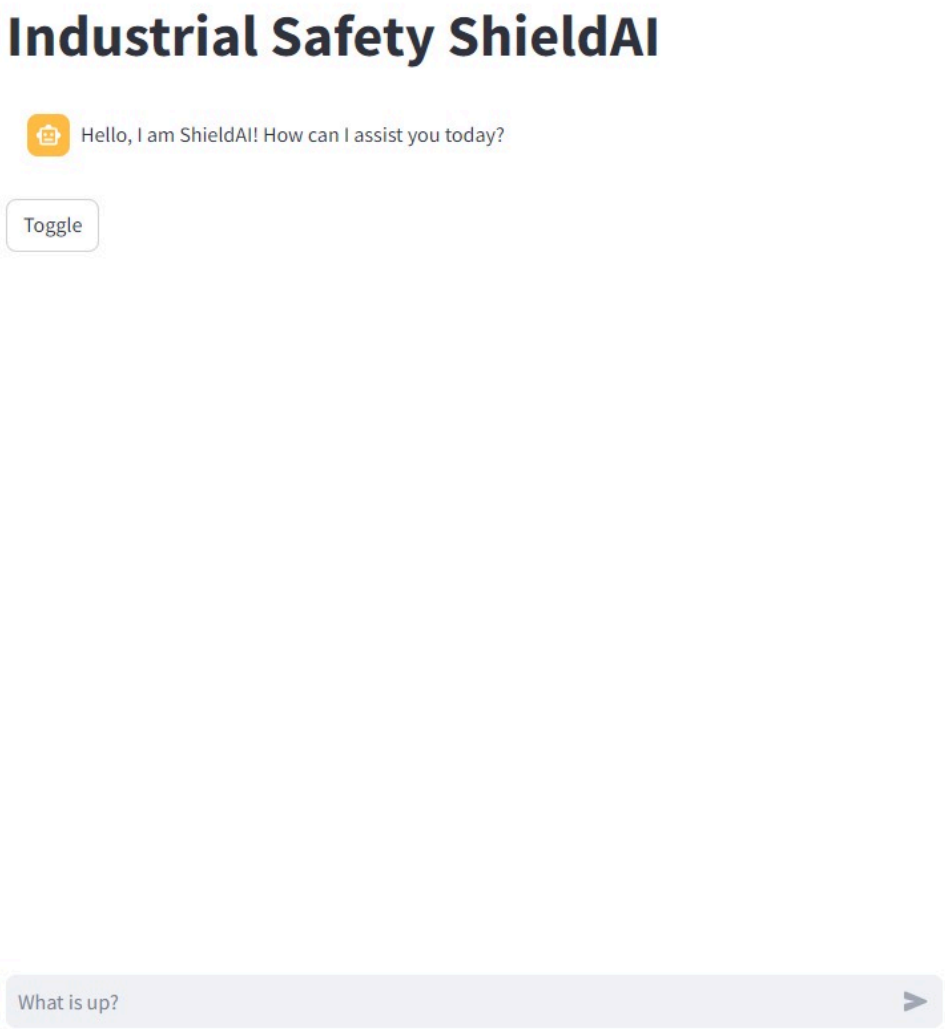
Cross Model Performance

	Model	accuracy	precision	recall
0	Tokenizer + Glove + RandomForestClassifier	0.750000	0.562500	0.750000
1	Tokenizer + Glove + DecisionTreeClassifier	0.535714	0.557341	0.535714
2	Tokenizer + Glove + SVC_Classifier	0.750000	0.562500	0.750000
3	Tokenizer + Glove + XG_Boost	0.726190	0.557927	0.726190
4	Tokenizer + Glove + SMOTE + RandomForestClassi...	0.678571	0.652778	0.678571
5	Tokenizer + Glove + SMOTE + DecisionTreeClassi...	0.357143	0.537698	0.357143
6	Tokenizer + Glove + SMOTE + SVC_Classifier	0.726190	0.557927	0.726190
7	Tokenizer + Glove + SMOTE + XG_Boost	0.607143	0.633815	0.607143
8	Transformers + RandomForestClassifier	0.750000	0.562500	0.750000
9	Transformers + DecisionTreeClassifier	0.500000	0.543103	0.500000
10	Transformers + SVC	0.750000	0.562500	0.750000
11	Transformers + XG_Boost	0.750000	0.569277	0.750000
12	Hypertuned RandomForestClassifier	0.654762	0.653571	0.654762
13	Hypertuned DecisionTreeClassifier	0.333333	0.523740	0.333333
14	Hypertuned SVC	0.666667	0.552632	0.666667
15	Hypertuned XG_Boost	0.607143	0.644534	0.607143
16	Neural Network	0.750000	0.750000	0.750000
17	BiLSTM with Glove + Tokenizer Embeddings	0.738095	0.738095	0.738095
18	BiLSTM with Sentence Transformer Embeddings	0.750000	0.750000	0.750000

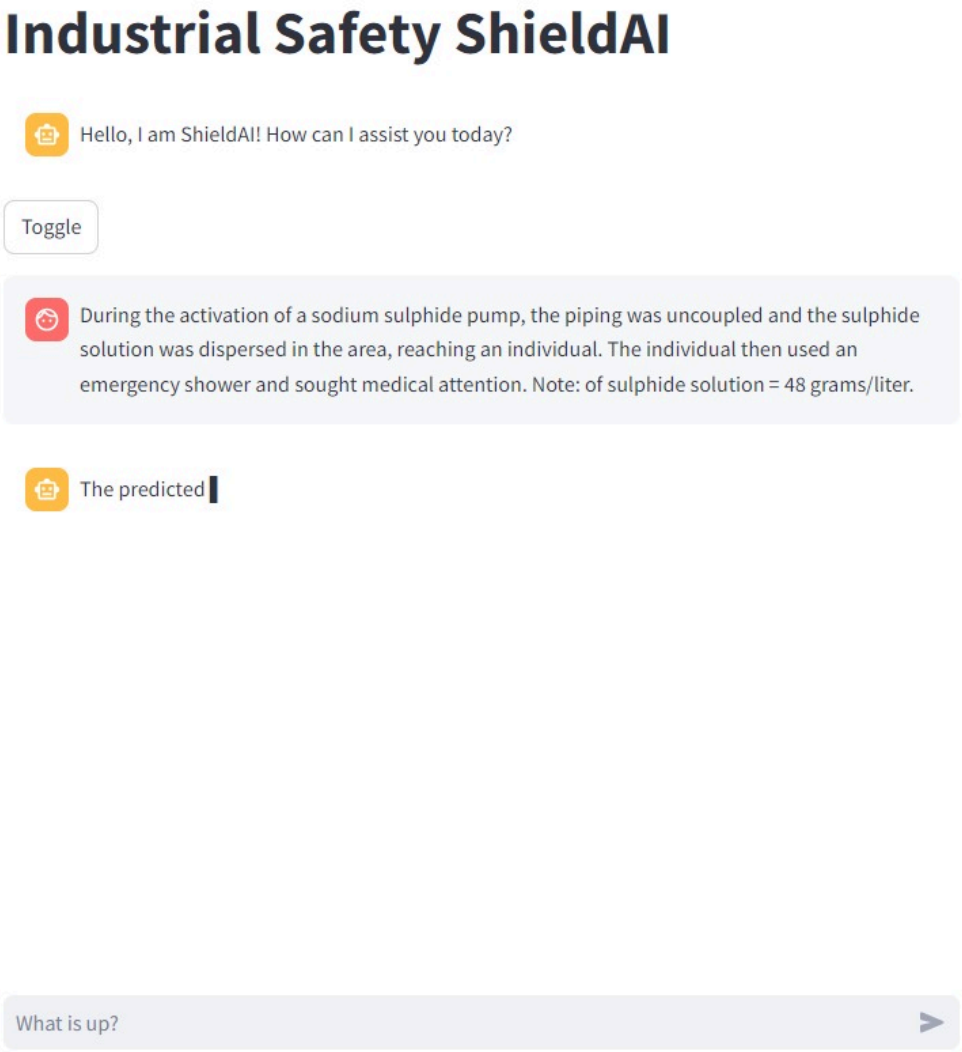
Milestone 3

Chatbot UI

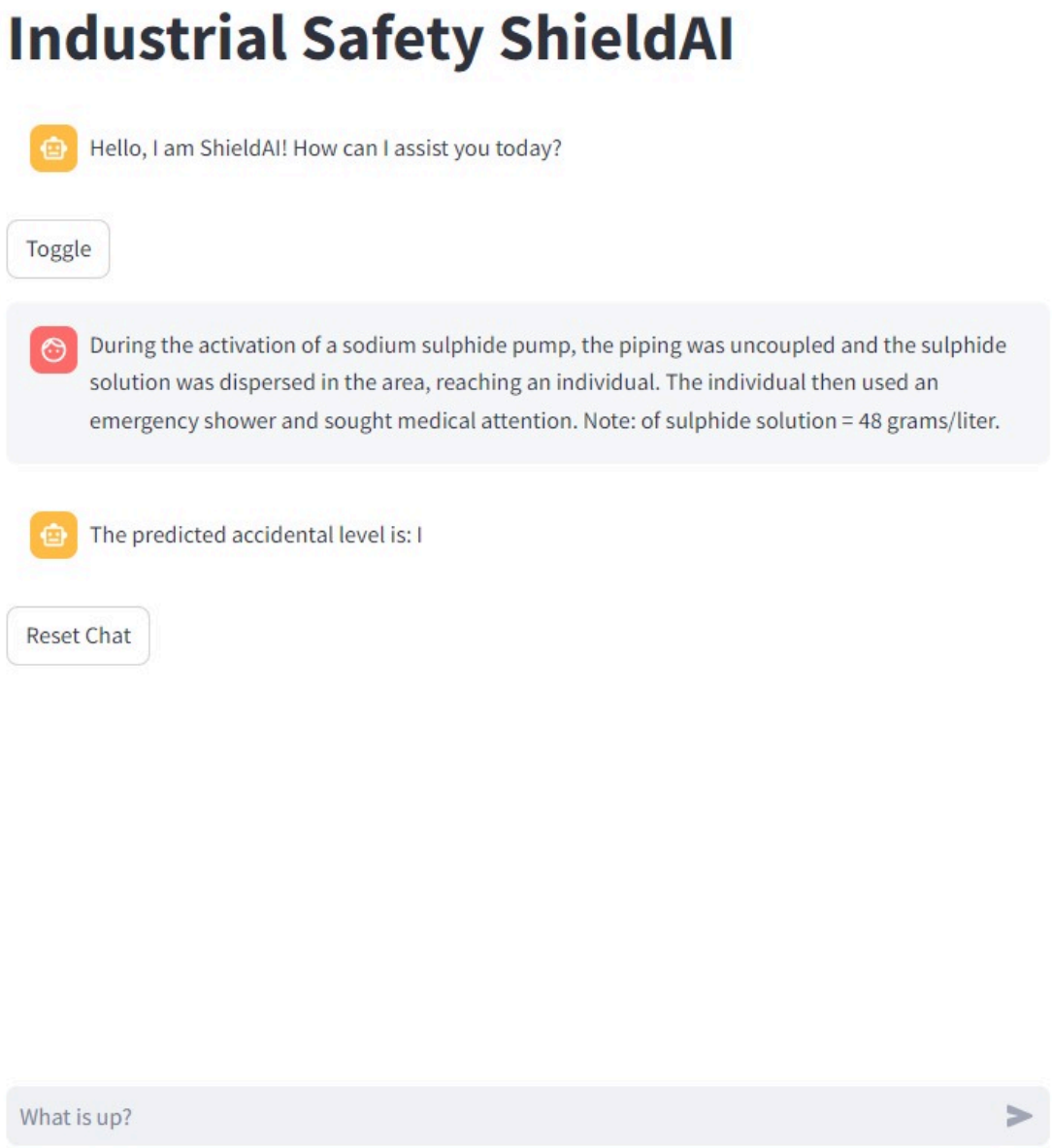
1. We get an introductory message and are given the option to type the description for prediction.



2. We share the description to predict accident level:



3. The chatbot makes a prediction for the accident level:



Summary

Our Streamlit-based chatbot provides an intuitive and user-friendly interface for predicting accident levels from textual descriptions. Here's a breakdown of the UI components and their functionalities:

Title and Introduction:

The app opens with a clear and concise title: Industrial Safety ShieldAI, and a greeting.

User Input Section:

Text Area: A prominent text area is provided for users to input the accident description or summary. This is where users describe the incident in detail.

Submit Button: An icon that looks like 'send button' allows users to submit their description for analysis.

Prediction Display:

Once the user submits the description, the app processes the input and displays the predicted accident level.

Additional Information:

We have added a reset button to reset the conversation and start afresh.

Implication - Business Values Derived

While Neural Network shows strong performance metrics, BiLSTM's use of Sentence Transformer embeddings offers several advantages in terms of embedding quality, consistency, potential for generalization, flexibility, and ease of integration. These factors make **'BiLSTM with Sentence Transformer Embeddings'** a compelling option, especially if the application requires nuanced understanding and adaptability. **We recommend the BiLSTM with sentence transformer embeddings based on our business use-case and the following characteristics**:

1. Embedding Quality: Sentence Transformers are known for producing high-quality embeddings that capture semantic meaning effectively. This can be particularly beneficial if the task involves understanding the context and relationships between words in sentences.

2. Consistency in Training Metrics: BiLSTM with Sentence Transformer Embeddings shows consistent improvement in training accuracy, precision, and recall from 0.739521 to 0.750000. This indicates that the model is learning well and improving its performance over time.

3. Potential for Generalization: The model has the potential to generalize well to unseen data. This is crucial for real-world applications where the model needs to perform reliably on new inputs.

4. Flexibility and Adaptability: Sentence Transformer embeddings can be fine-tuned for various downstream tasks, making our final model highly adaptable.

5. State-of-the-Art Techniques: Sentence Transformers leverage state-of-the-art techniques in natural language processing (NLP), which can provide a competitive edge in terms of performance and accuracy. This can be particularly important in applications where precision and recall are critical.

6. Ease of Integration: Sentence Transformer models are often easier to integrate into existing NLP pipelines and frameworks. This can save time and effort in deployment and maintenance, making our final model a practical choice for production environments.

Limitations

As observed, the dataset has limited values and is highly skewed. Accident Level **I** dominates the dataset thereby making the prediction for other classes difficult for the model.

- **Insufficient Observations:** The current dataset lacks a sufficient number of observations to accurately analyze the causes of accidents. Increasing the number of observations will yield more reliable results.
- **Limited Features:** The dataset has a limited number of features, which restricts the depth of analysis. Expanding the range of features will enhance the model's predictive capabilities.
- **Data Quality Issues:** There is a lack of access to high-quality data, which hampers the accuracy and reliability of the analysis. Ensuring access to quality data is crucial for better outcomes.
- **Real-World Model Performance:** Our model may not perform as well in real-world scenarios as it does in controlled environments. Identifying and addressing these gaps is essential for practical application.
- **Production Performance:** Once deployed, the finalized model might achieve a lower F1-score compared to its performance during development. Continuous monitoring and adjustment are necessary to maintain high performance.
- **Accuracy in Predictions:** Given that we are predicting accident levels, achieving near-perfect accuracy is critical to prevent industrial accidents. Striving for 100% accuracy, or as close as possible, is imperative.
- **Enhancing the Solution:** To address these limitations, we need to focus on collecting more observations, expanding the dataset with additional features, ensuring access to high-quality data, and continuously monitoring and refining the model post-deployment.

Recommendations

1. In this project, we uncovered that the predominant causes of accidents are manual operation errors and time-related factors.
2. To significantly reduce the incidence of accidents, it is imperative to enforce more rigorous safety standards during high-risk periods.
3. We have come to realize that detailed accident information, such as the 'Description', is immensely valuable for analyzing the root causes.
4. By integrating more comprehensive data we can more precisely identify the causes of accidents.
5. Expanding the dataset beyond the current 425 records will enable us to input more data into ML, DL and transformer based models, thereby improving their training, evaluation, and overall performance.

While there are numerous critical risk descriptions, with the expertise of Subject Matter Experts (SMEs), we can detect outliers and gain a deeper understanding of the data.

Closing Reflections

We evaluated models on accuracy, computational efficiency, and ease of implementation. The Bi-Directional LSTM with Sentence Transformer Embedding, emerged as the top choice due to its superior accuracy and balanced performance across various metrics.

We saved the final model for future use, enabling quick deployment and re-evaluation without the need for retraining.

We then loaded the pickled model and used it to make predictions on new data, to showcase consistent performance on real-world production data.

Technical References:

1. Sentence Transformer: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
2. GloVe: http://nlp.stanford.edu/data/glove.6B.zip
3. Scikit Learn: https://scikit-learn.org/stable/